



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduction

Information Security

Prof. Stefano Ferretti

Outline

- Part I: Few foundational terms and concepts
- Part II: Some use cases
- Part III: Lessons learned from use cases



Part I: Foundational Terms we Need



Can I give these terms for granted?

- Scalability
- Availability
- Single point of failure
- Bottleneck
- Integrity
- Confidentiality
- Privacy
- Fairness
- Auditability
- Client/Server model
- Peer-to-Peer model
- Churn



Maybe, we can focus on a subset

- Scalability
- Availability
- Single point of failure
- Bottleneck
- **Integrity**
- **Confidentiality**
- **Privacy**
- **Fairness**
- Auditability
- Client/Server model (**some issues only**)
- Peer-to-Peer model
- Churn



Integrity

Data integrity is the assurance that **information remains accurate, complete, and consistent** throughout its entire lifecycle

- Data not altered or destroyed in an unauthorized or accidental manner

Integrity in ML

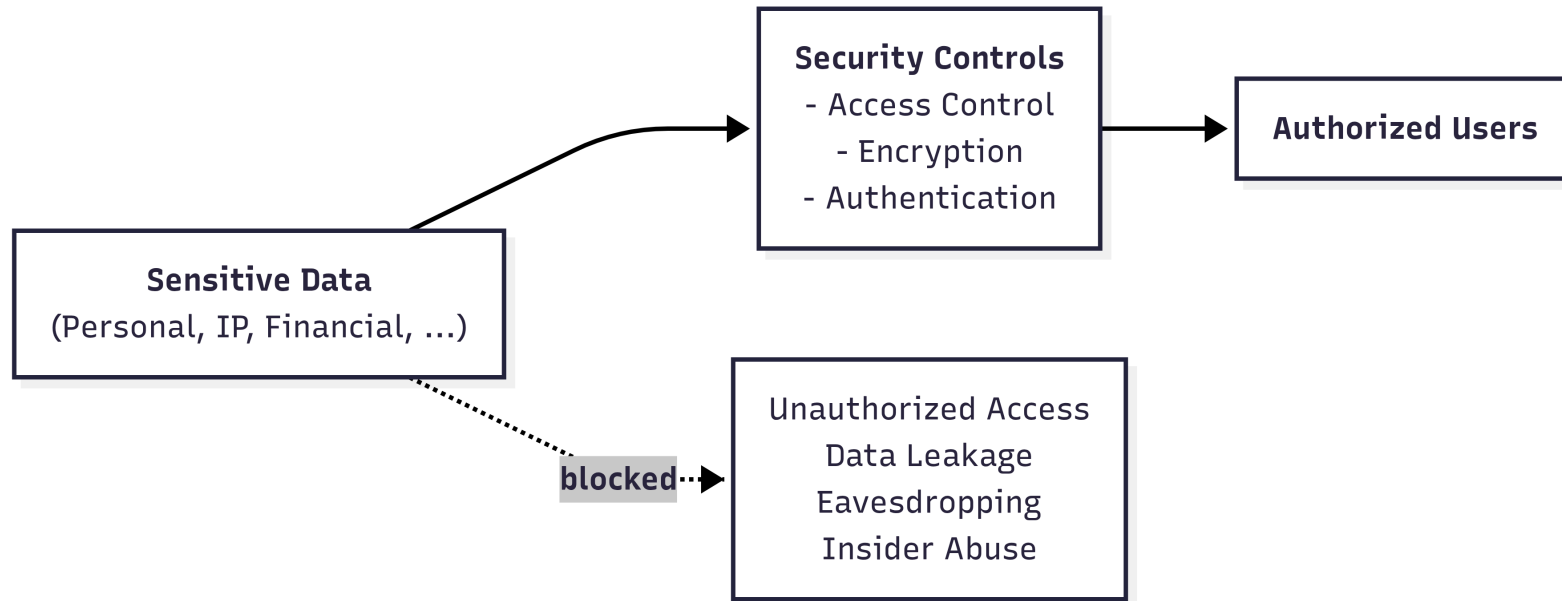
- **no data poisoning:** need to ensure **training data** is not altered
- **model integrity:** need to ensure **model parameters** are not altered



Confidentiality

Confidentiality ensures sensitive data is accessed only by authorized individuals

- Ensures that only authorized entities can access sensitive information
- Protects data from unauthorized disclosure or exposure



Privacy

Privacy is the right of individuals to exercise control over how their **personal data is collected, processed, and shared**, ensuring the confidentiality, integrity, and ethical use of their information

- **Privacy in Storage:** store only necessary data (**data minimization**), properly encrypted (encryption at rest)
- **Privacy in ML:** individuals not re-identified from model outputs (**membership inference protection**)



Privacy vs Confidentiality

- **Confidentiality** → access control
- **Privacy** → inference control



Fairness

Fairness ensures that information systems and algorithms operate without creating or reinforcing bias. It implies the **equitable treatment of all users and data subjects**, preventing discriminatory outcomes and ensuring balanced resource allocation across the network

- **Fairness in distributed systems:** fair resource allocation, load balancing, latency equity
- **Fairness in ML:** bias mitigation, group fairness, algorithmic transparency



The problem of trust



Central Trusted Authority

What we **trust**

- **services**
- **data sources**
- **data**

However, in modern systems, **trust cannot be assumed**



Two Fundamental Objectives

Systems should be analyzed along two axes (**besides performance**):

- **Trustworthiness**
 - Can we rely on data, models, and system behavior?
- **Privacy**
 - Can we use data without exposing sensitive information?

These are **not independent** and often introduce trade-offs



Trust vs Privacy: A Tension

Improving one objective may weaken the other

Examples:

- Logging improves **auditability** but may reduce **privacy**
- Data sharing improves **utility** but increases **exposure risk**
- Differential Privacy improves privacy but reduces model accuracy



Does the *AI* revolution change the overall picture?

In terms of trustworthiness, privacy, scalability, fairness, auditability, etc.



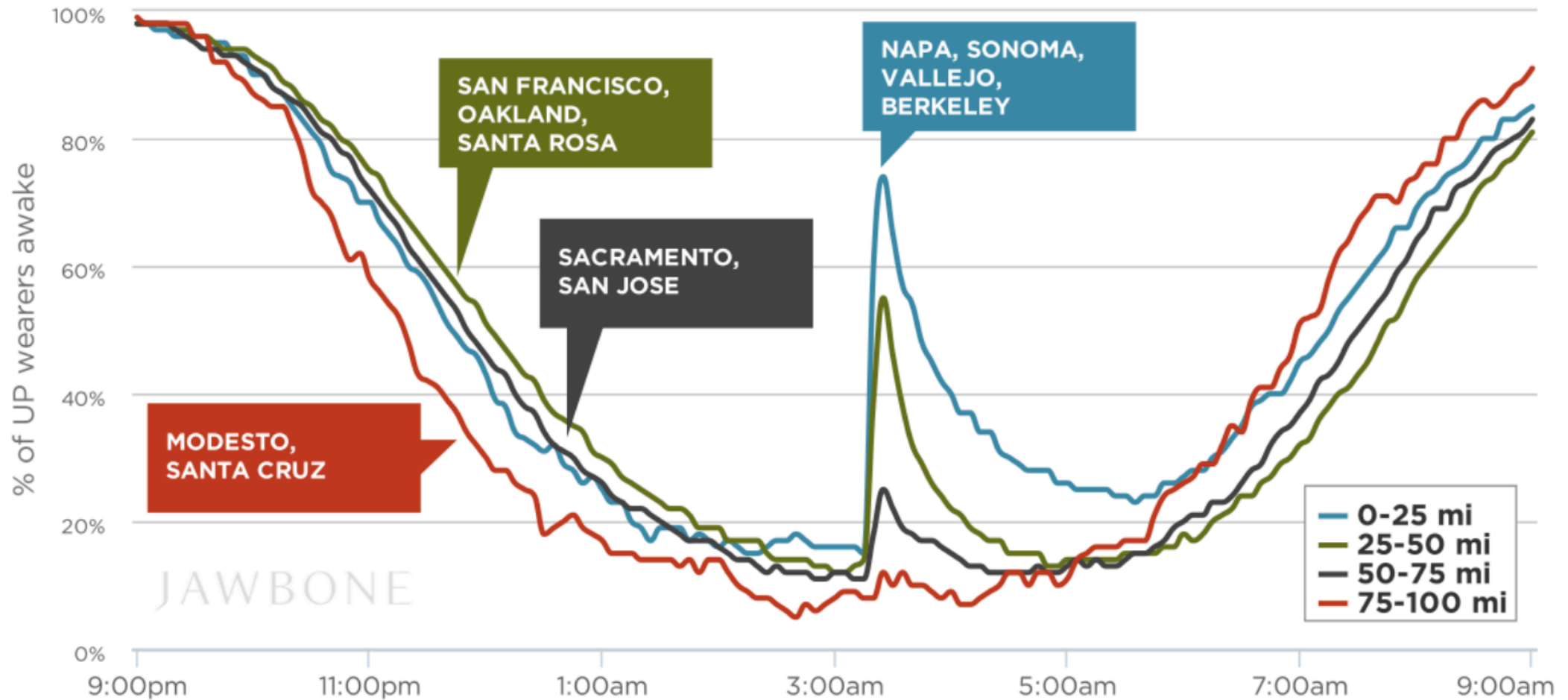
Does the AI revolution change the overall picture?

In terms of trustworthiness, privacy, scalability, fairness, auditability, etc.

Trustworthy AI



The landscape has changed



The landscape has changed

Open Source Intelligence (OSINT)

More data → More knowledge

- Correlation
- Patterns
- Insights

Aggregation is **power**

More data → More exposure

- Inference
- Re-identification
- Hidden sensitivity

Aggregation is **risk**



Real-world example

Netflix Prize Dataset (2006)

- “Anonymized” movie ratings released
- No names, no direct identifiers



- Cross-referenced with IMDb ratings
- Users re-identified



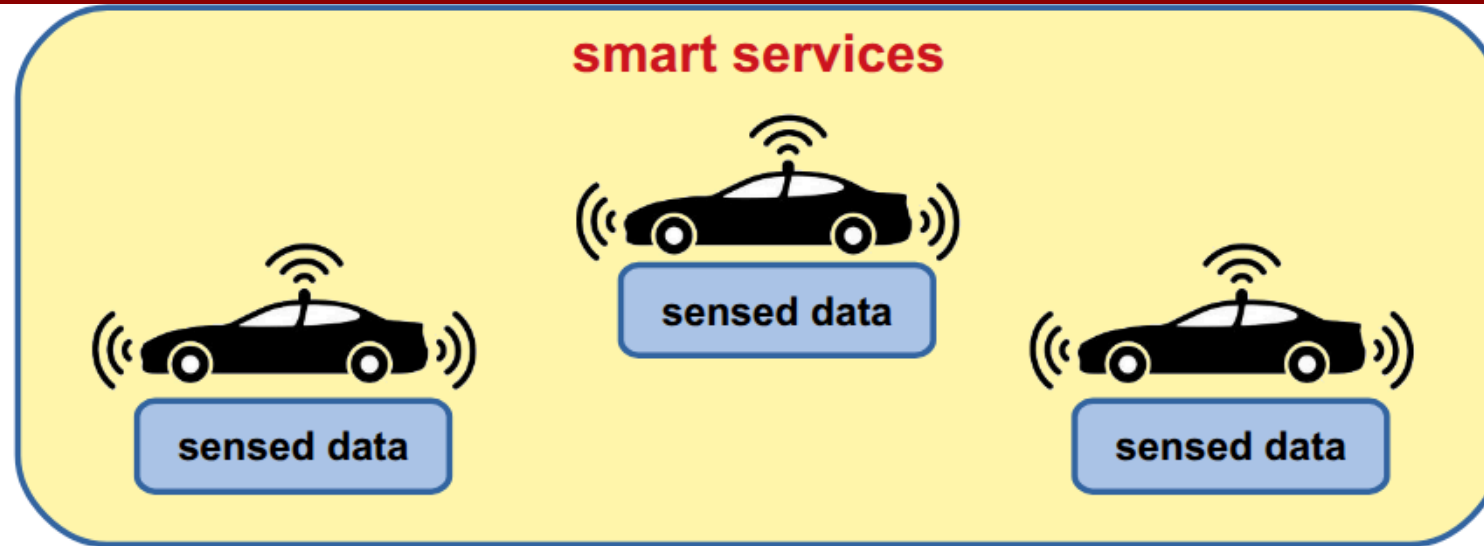
Part II: Some Use Cases



Use Case #1: Crowdsensing Applications (in Decentralized Systems)



Smart Transportation Systems



Smart Transportation Systems

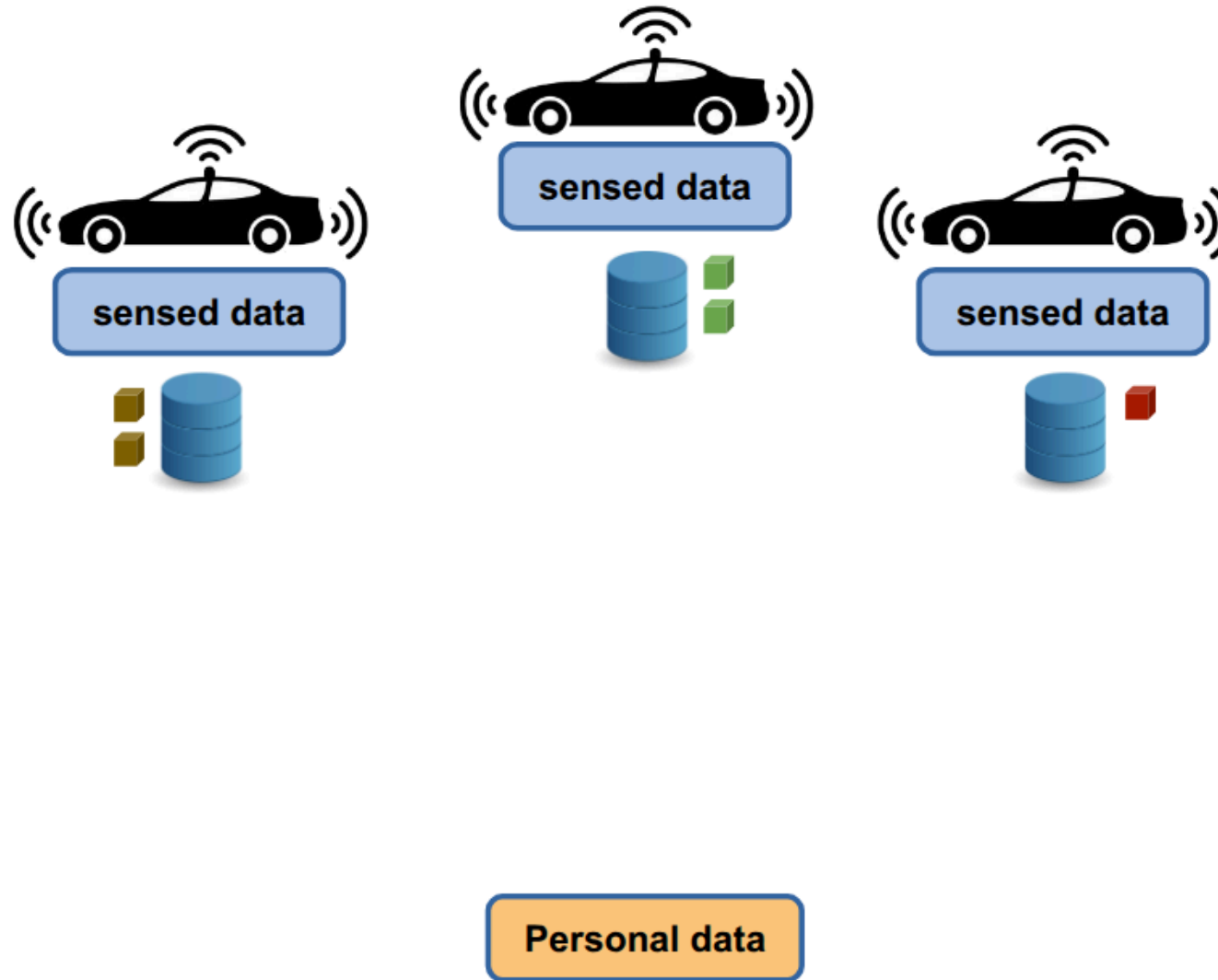
The system implements **crowdsensing-as-a-Service**

Data used to provide:

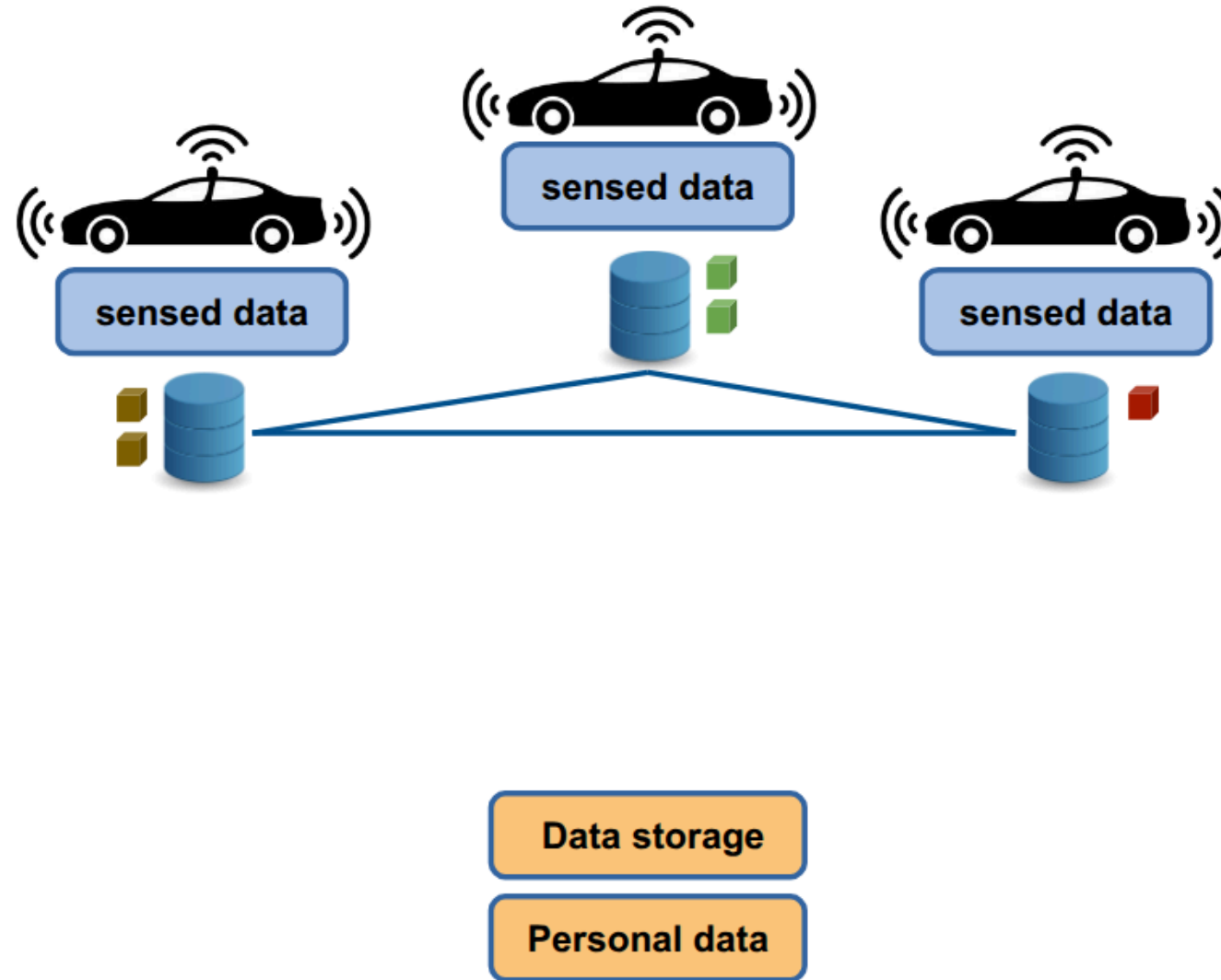
- Traffic analysis
- Route optimization
- Safety alerts (e.g., crash, wrong-way driving, weather conditions)



Smart Transportation Systems



Smart Transportation Systems



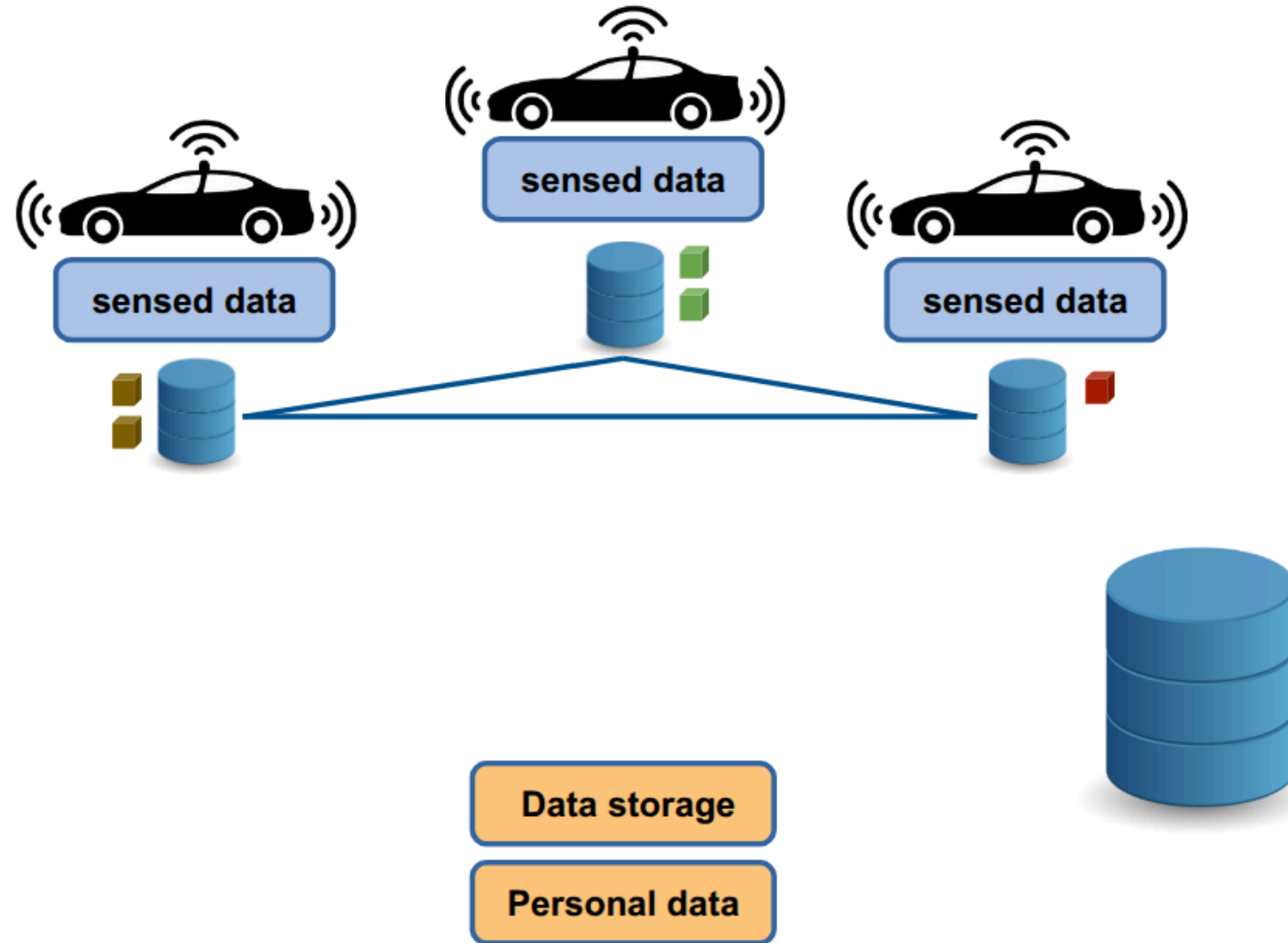
Security Objectives

The system must guarantee:

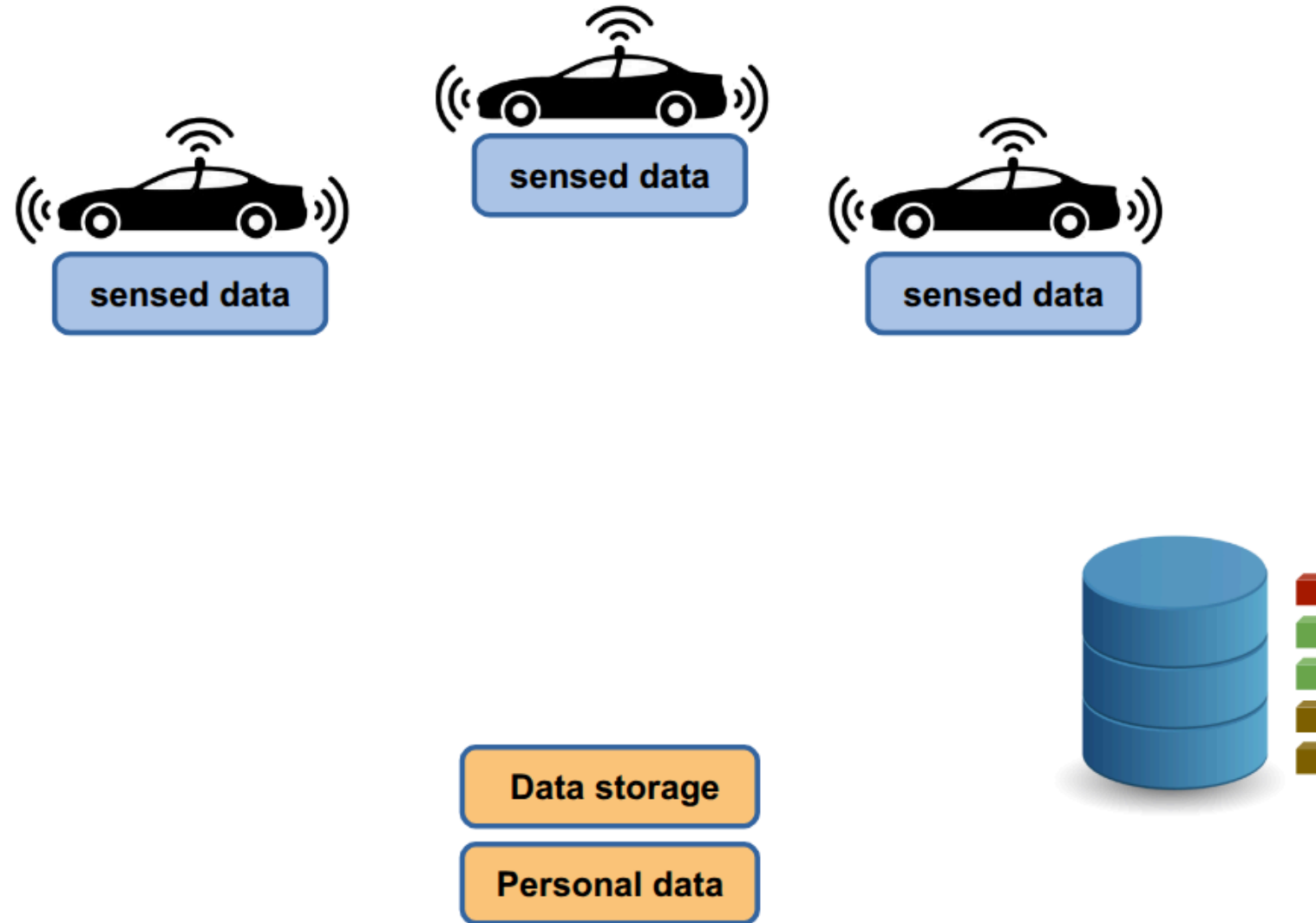
- **Integrity** → Data cannot be altered undetectably
- **Confidentiality** → Unauthorized parties cannot access raw data
- **Availability** → Data remains retrievable when needed
- **Authenticity** → Data provenance is verifiable
- **User Sovereignty** → Data subjects retain control



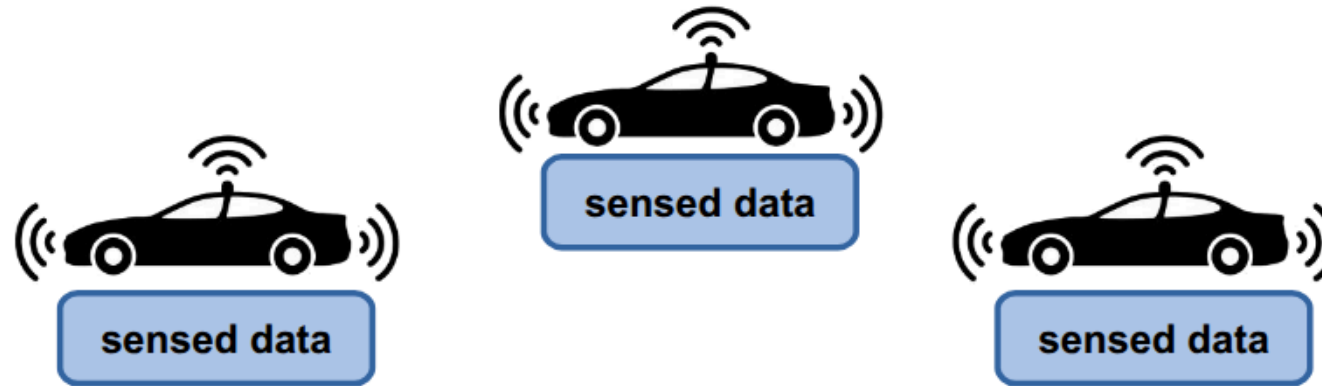
Opt1: central entity maintains crowdsourced data



Opt1: central entity maintains crowdsourced data



Opt1: central entity maintains crowdsourced data



Users **lose the sovereignty** over their data

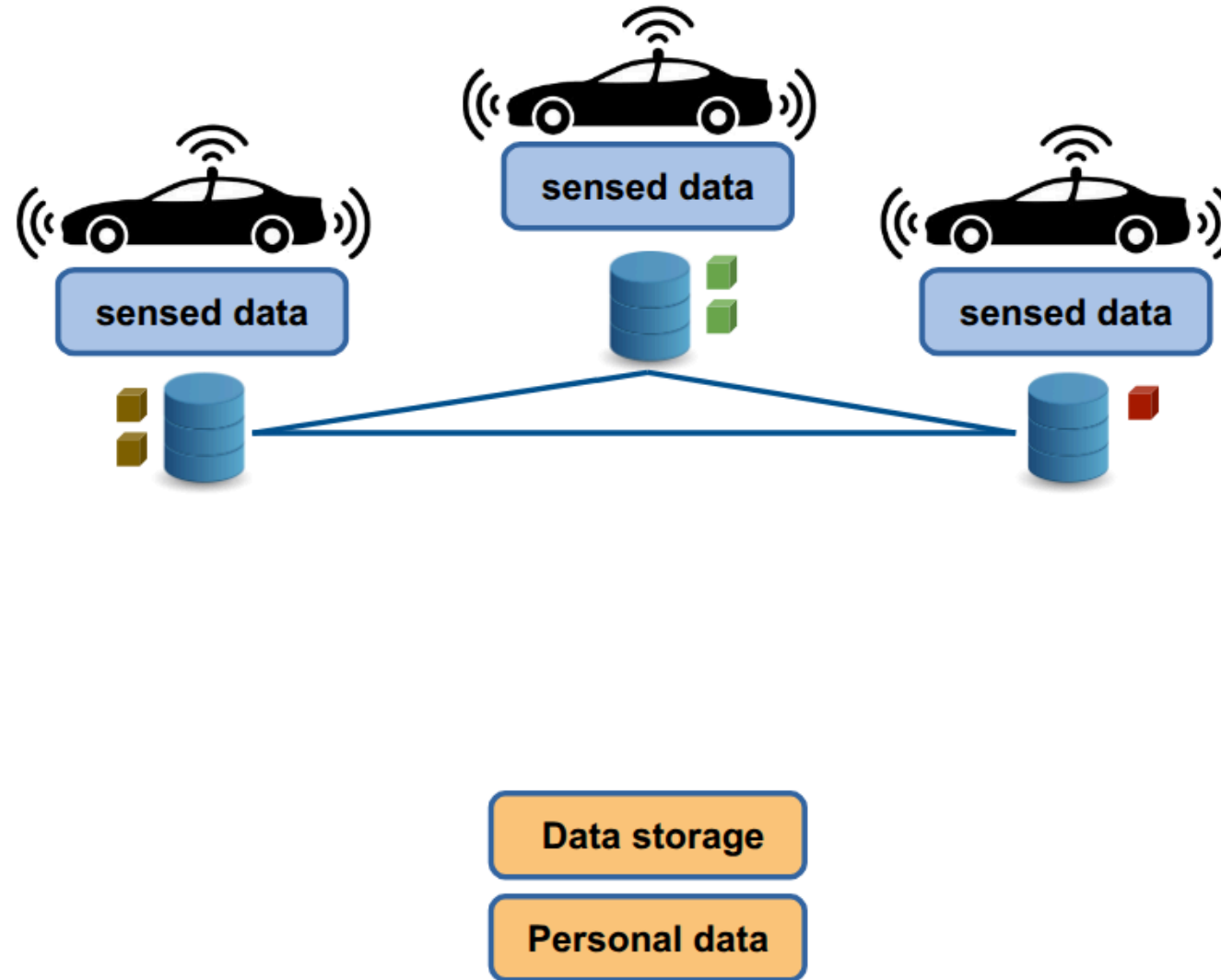
- data controller gains all the data
- data controller can alter the data
- users must rely on the controller

Data storage

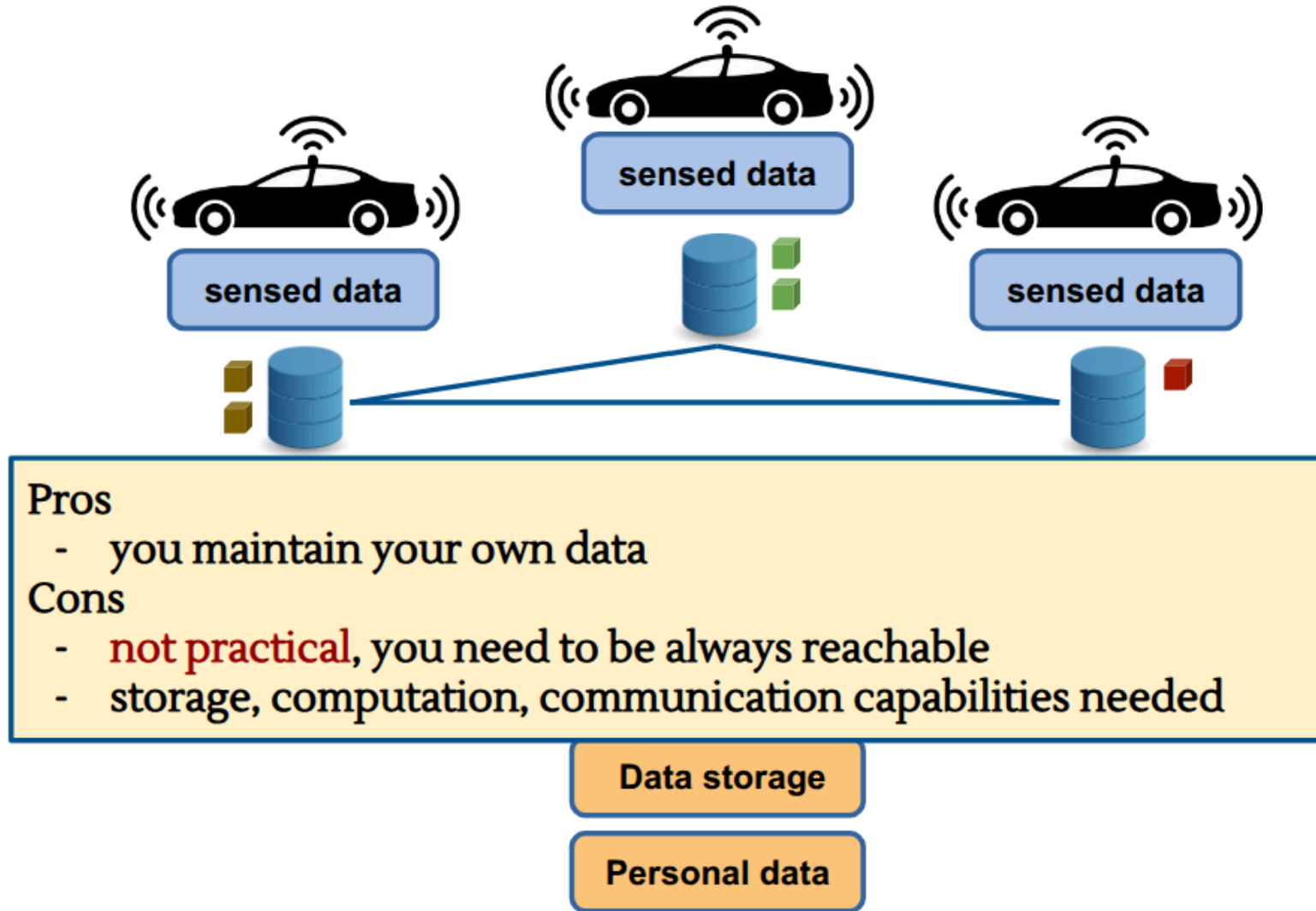
Personal data



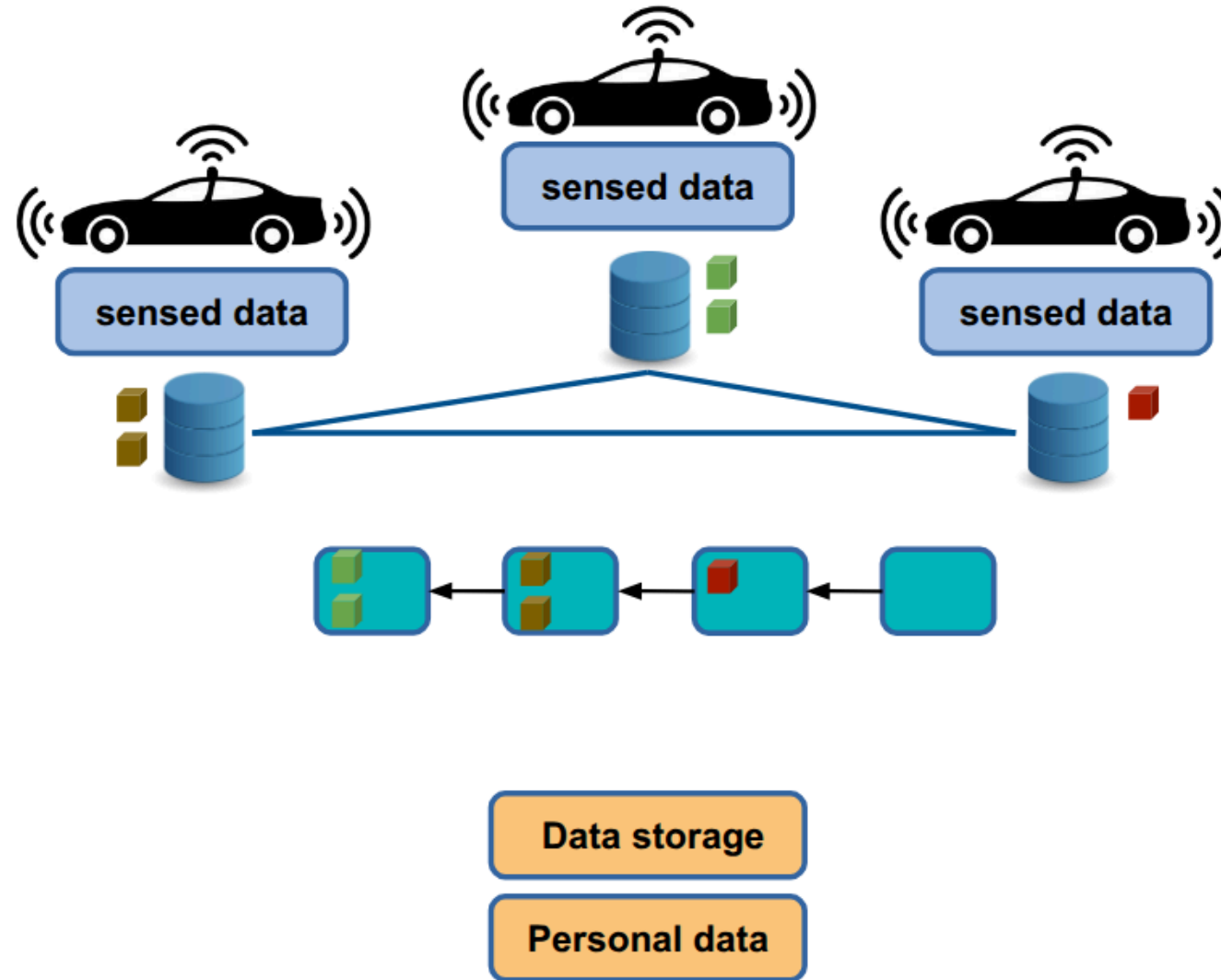
Opt2: keep data locally - distribute upon request



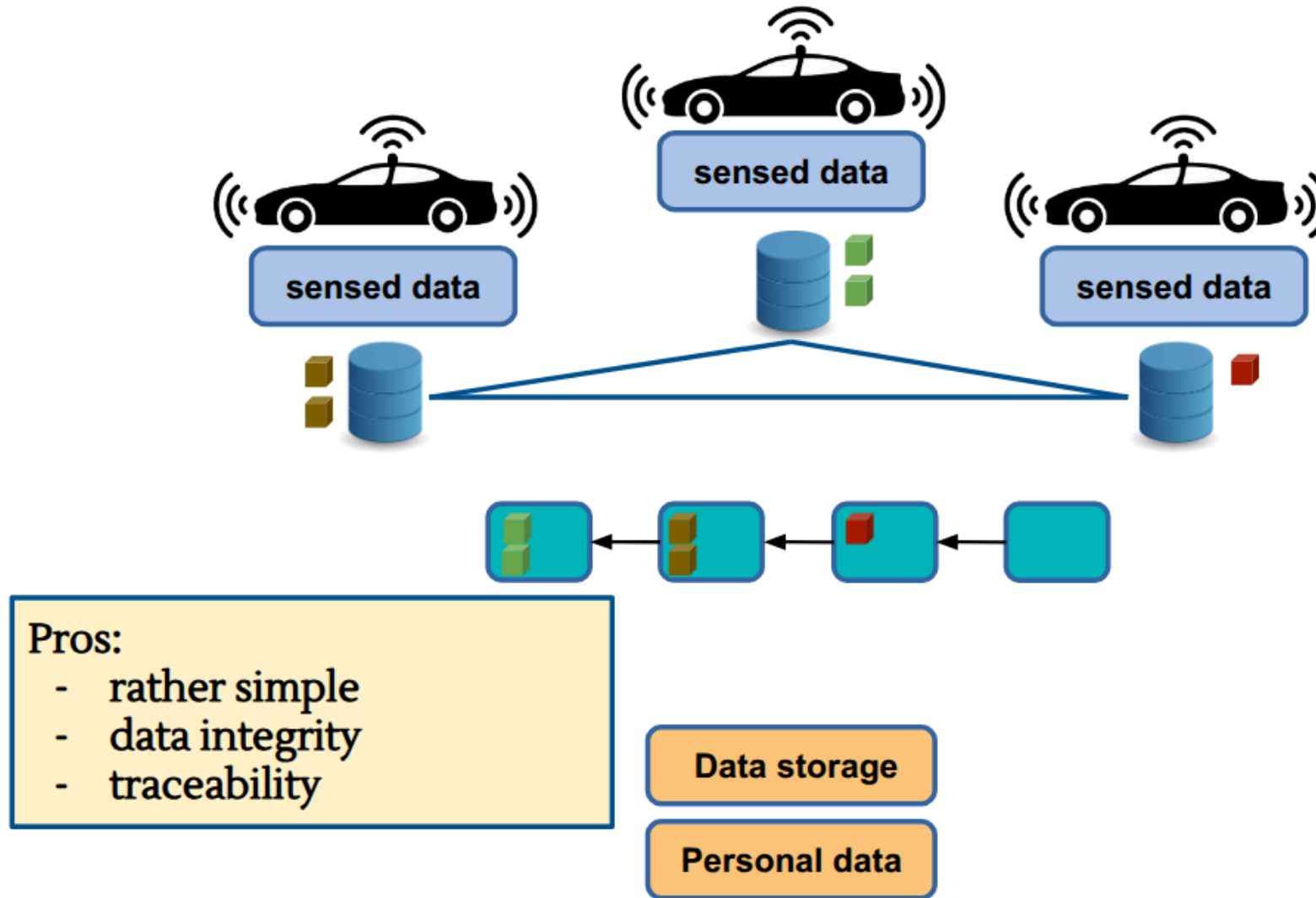
Opt2: keep data locally - distribute upon request



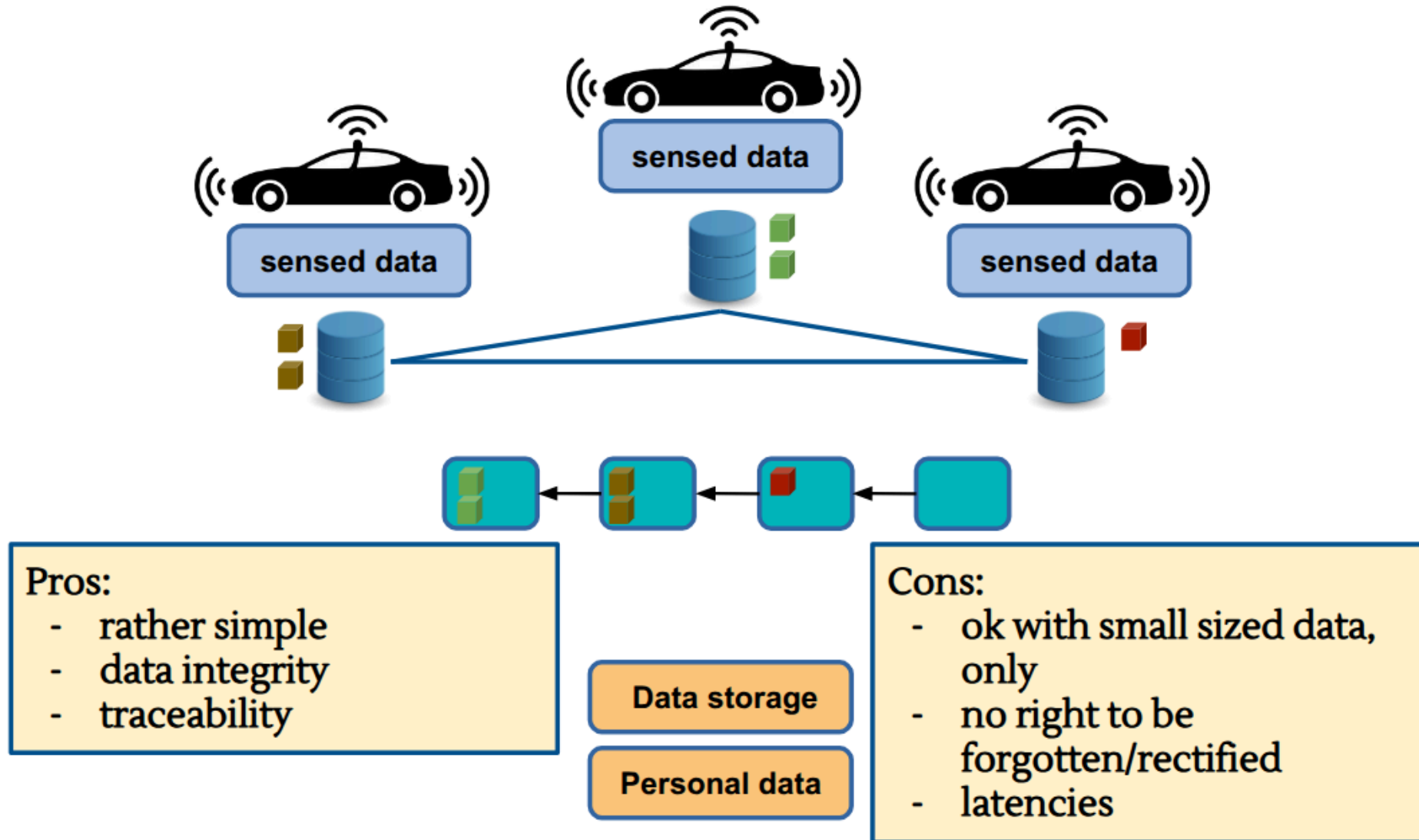
Opt3: use a ledger to register data



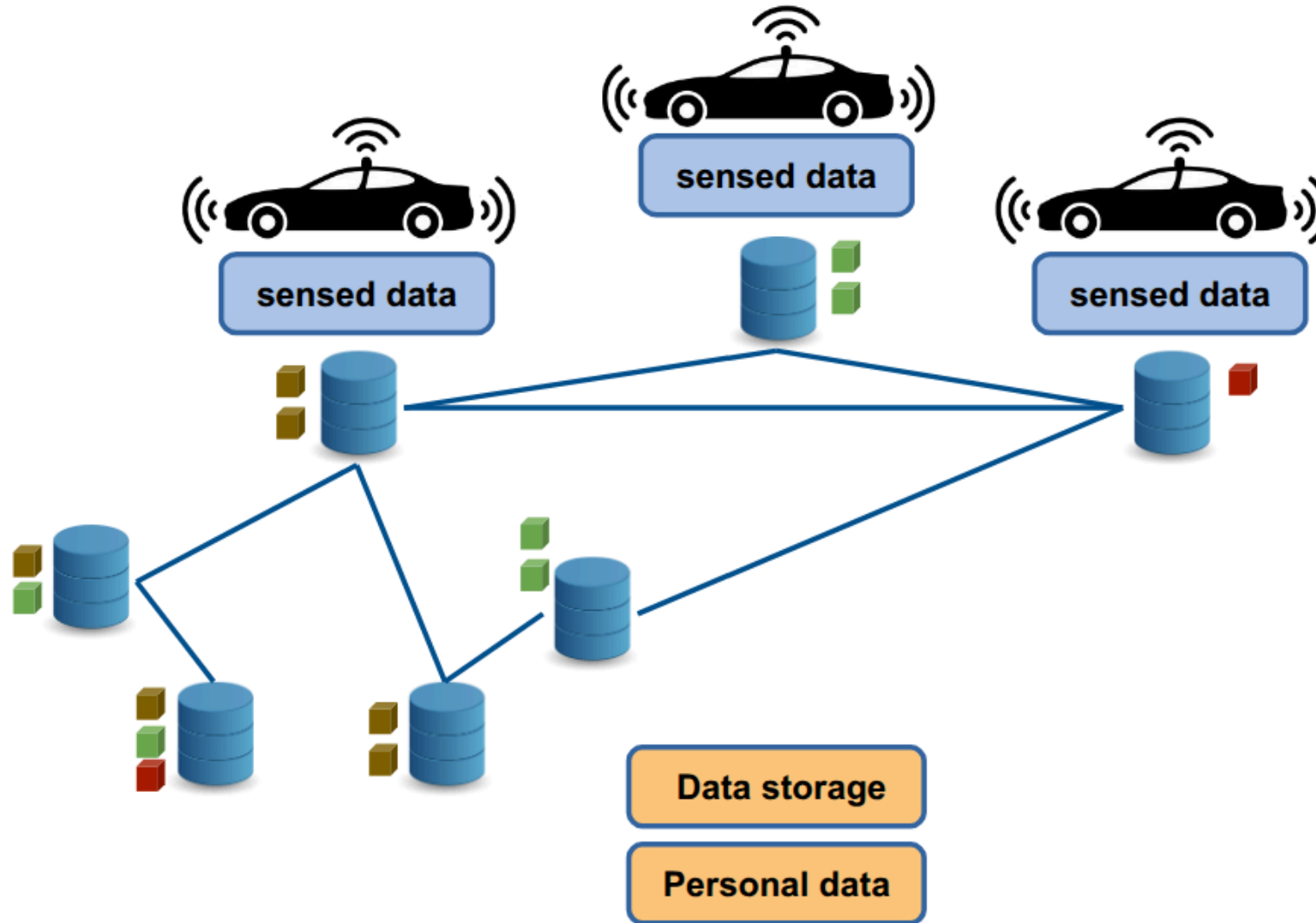
Opt3: use a ledger to register data



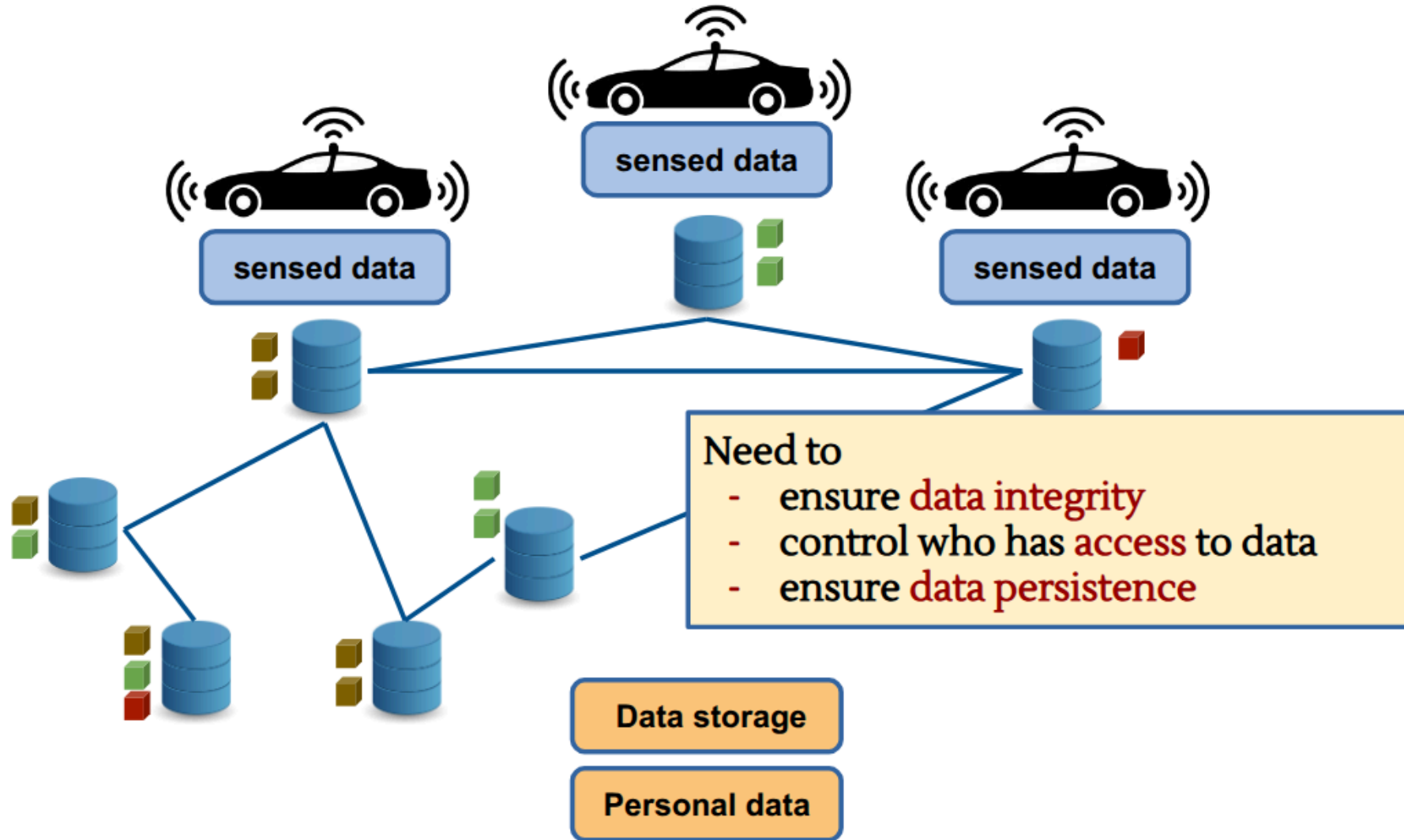
Opt3: use a ledger to register data



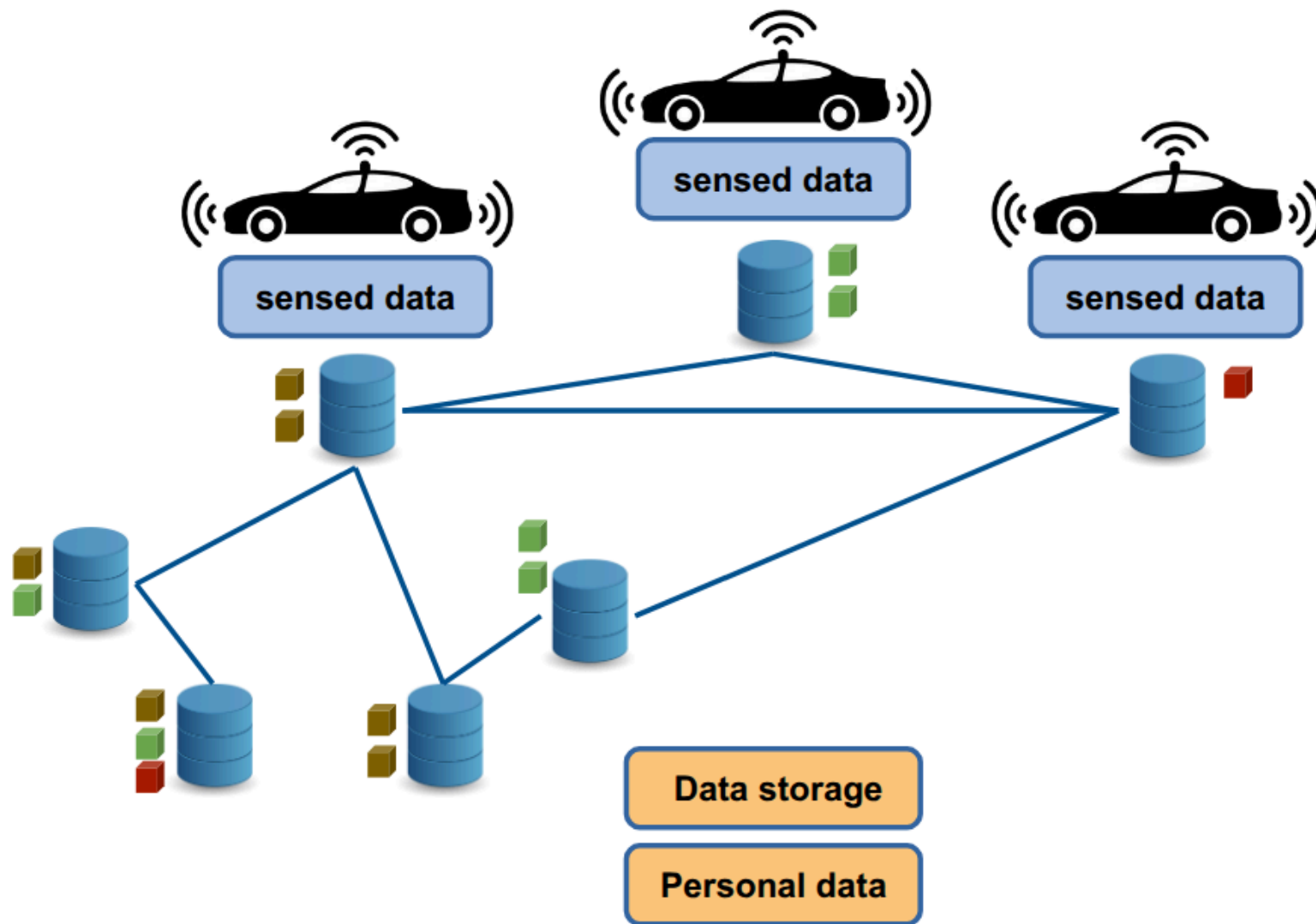
Opt4: use decentralized file systems



Opt4: use decentralized file systems



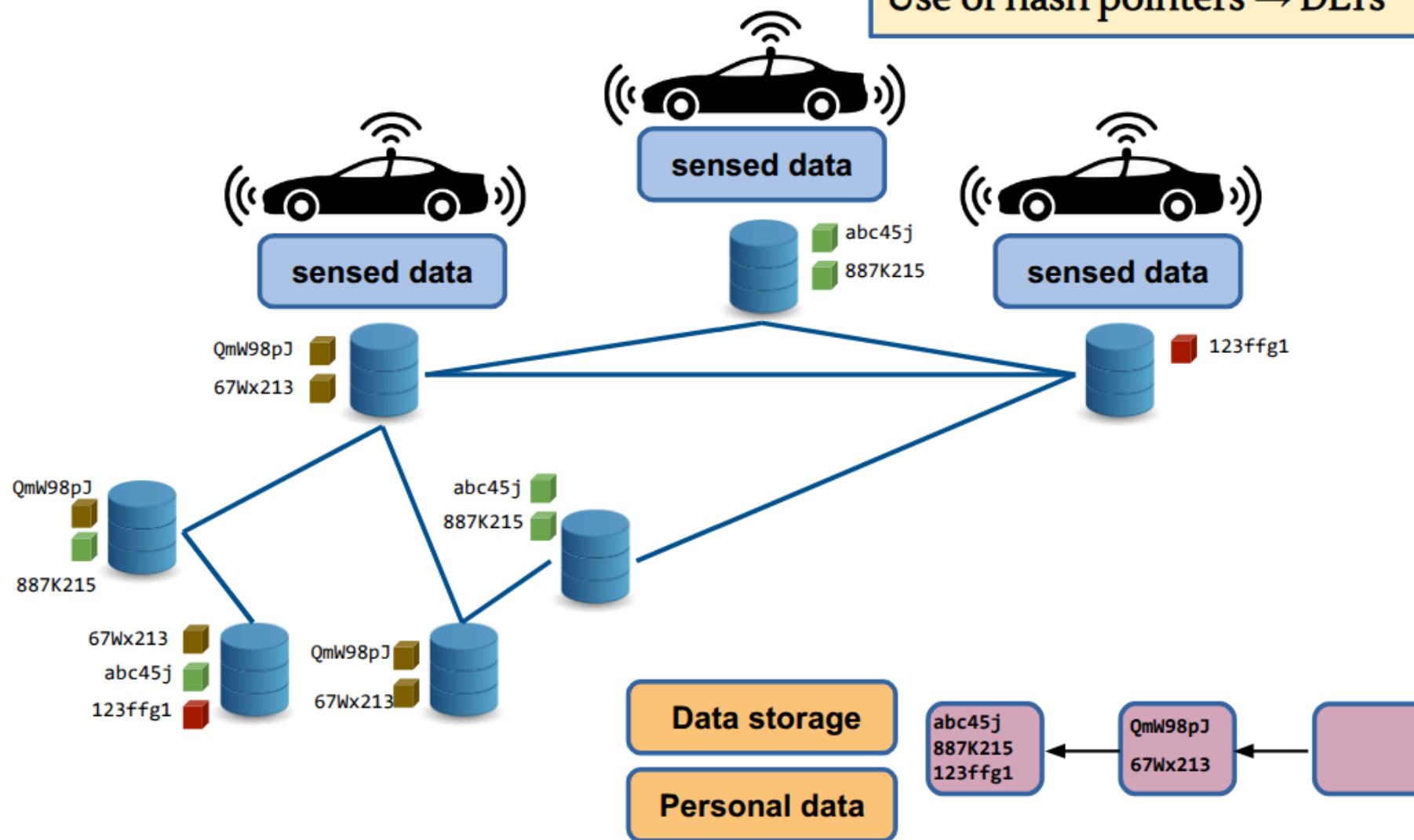
Data Integrity



Data Integrity

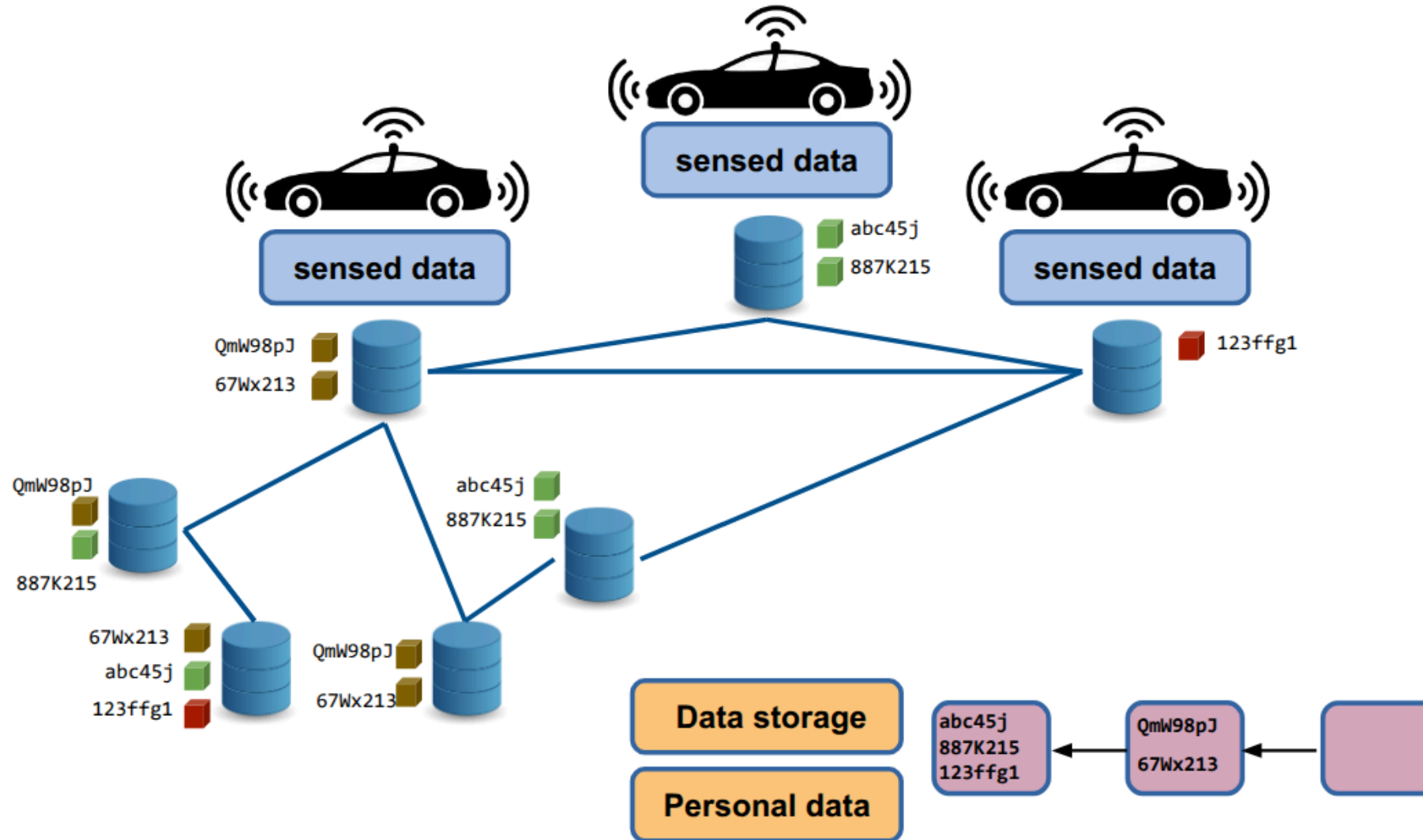
Content based addressing
(instead of location based)

Use of hash pointers → DLTs

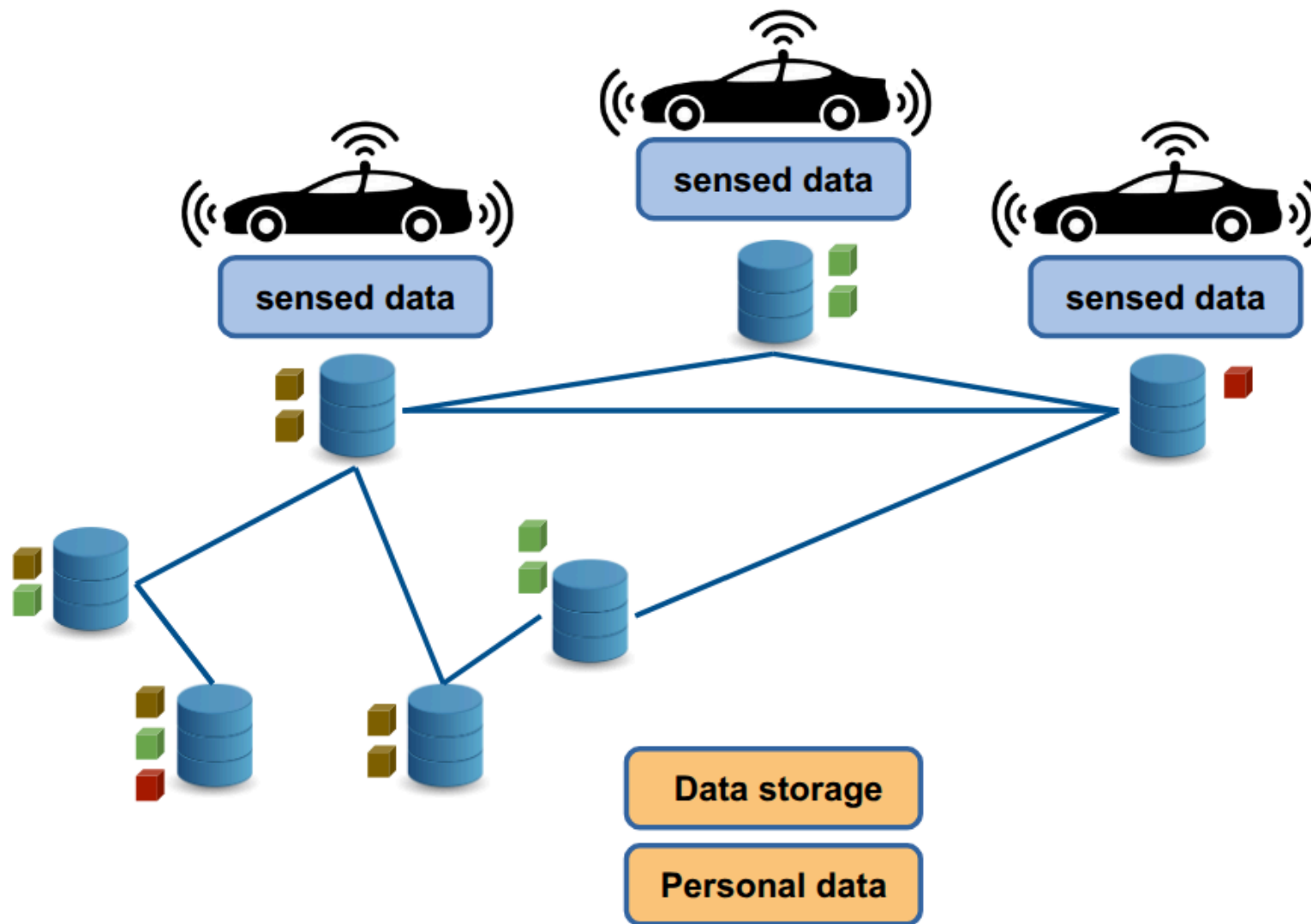


Data Integrity

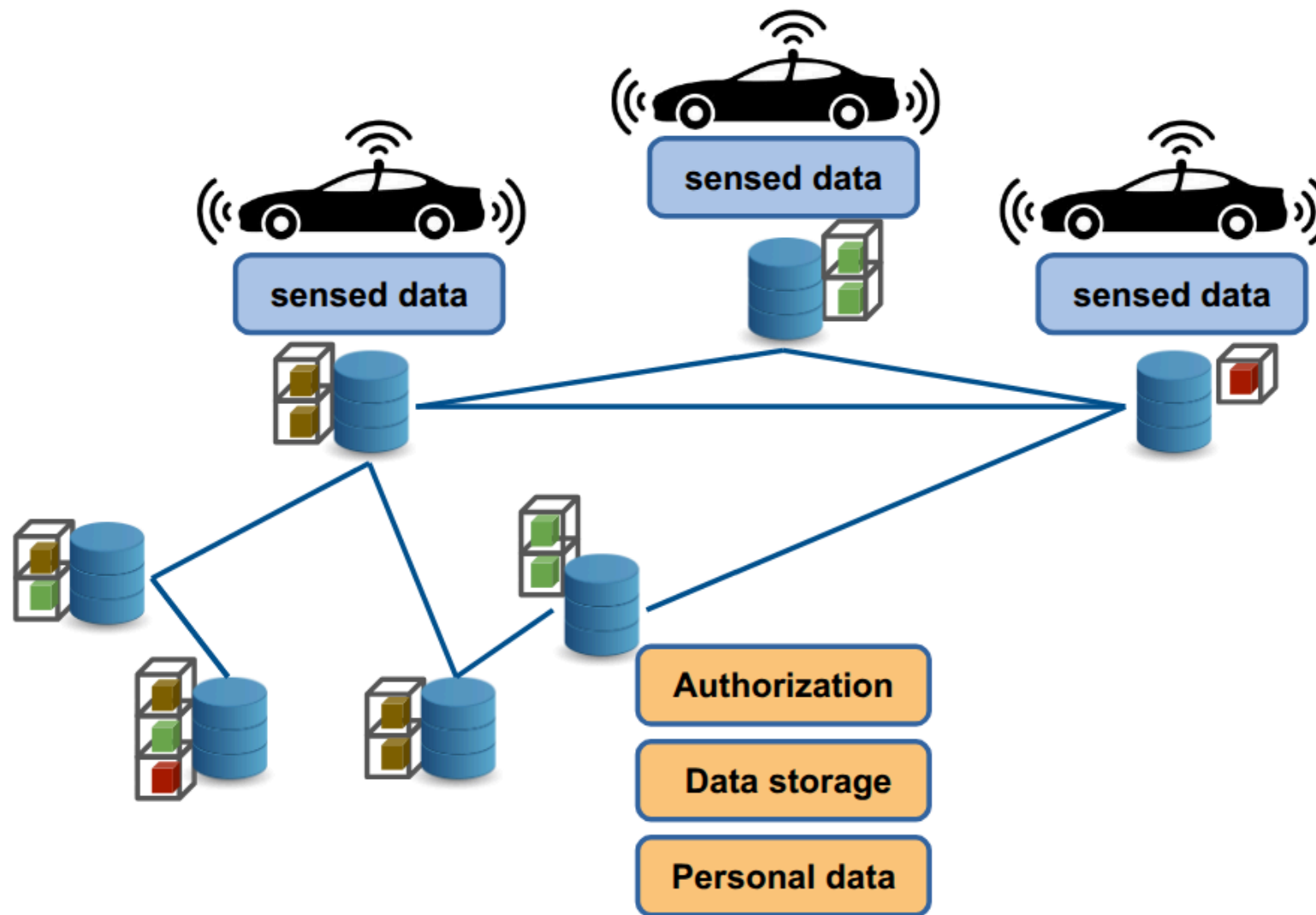
- Possible to remove/modify data
- Fast data upload to DFS
- Latencies to upload hashes less problematic



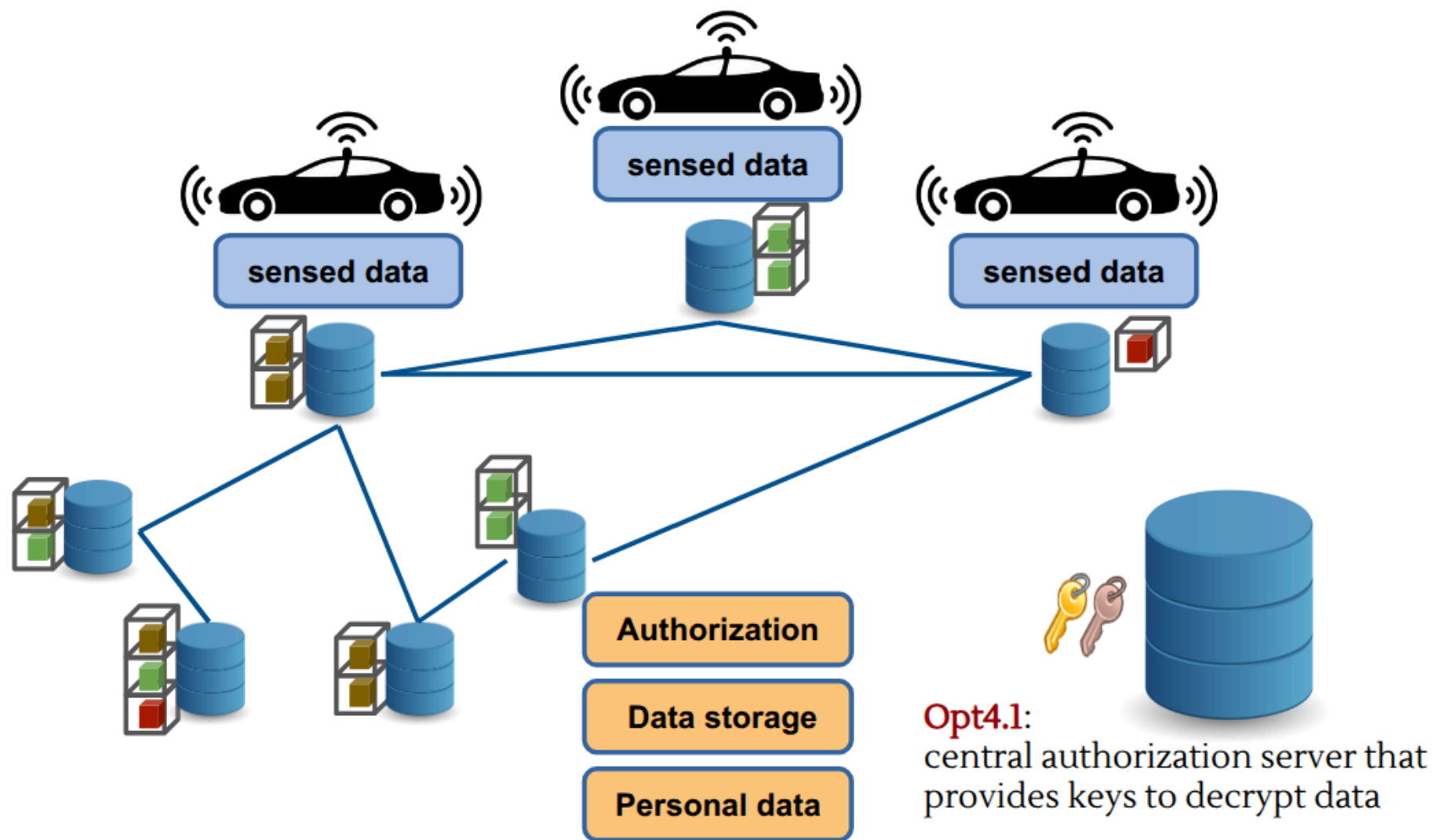
Access Control?



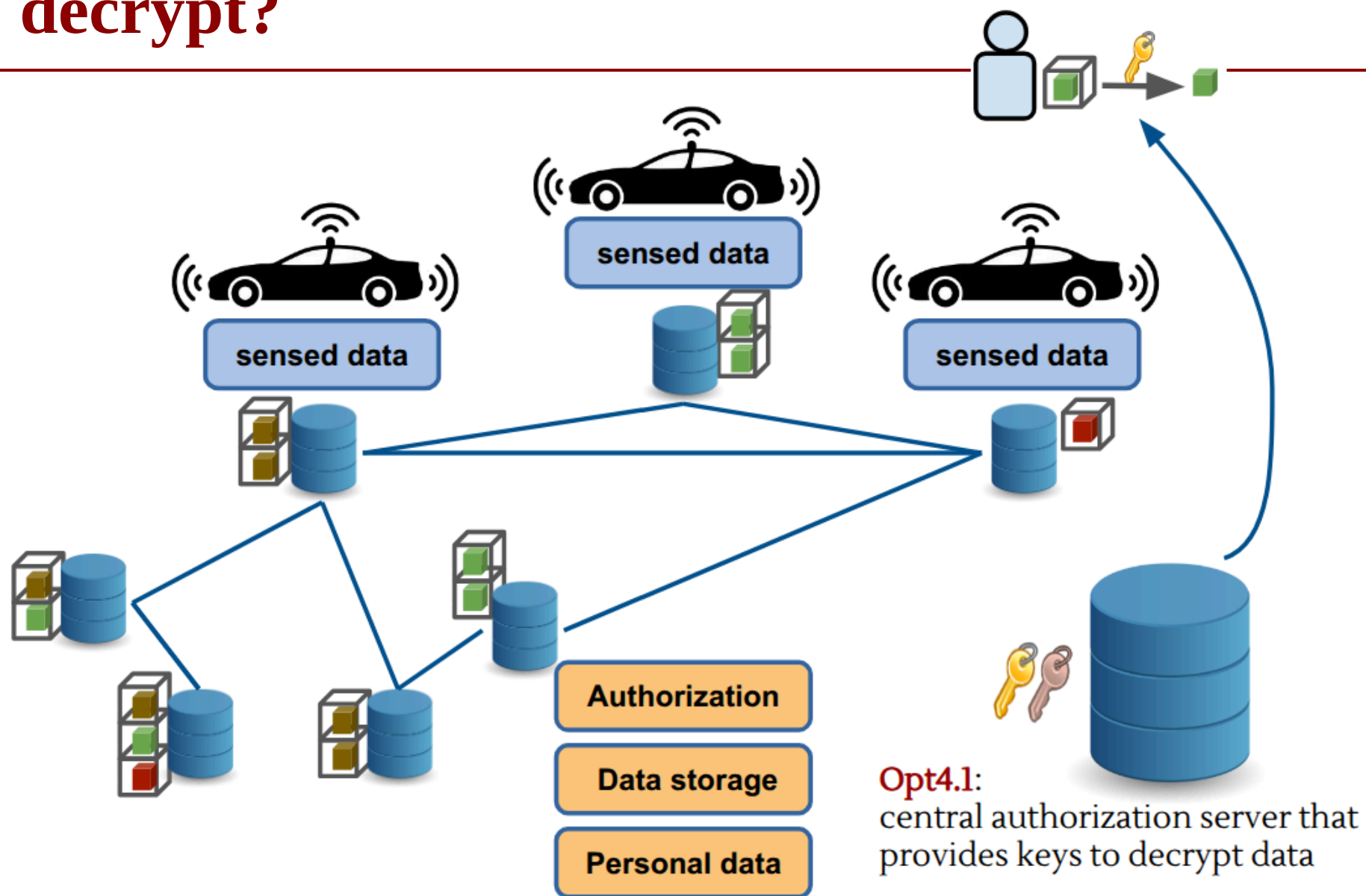
Access Control: DFS + Encryption



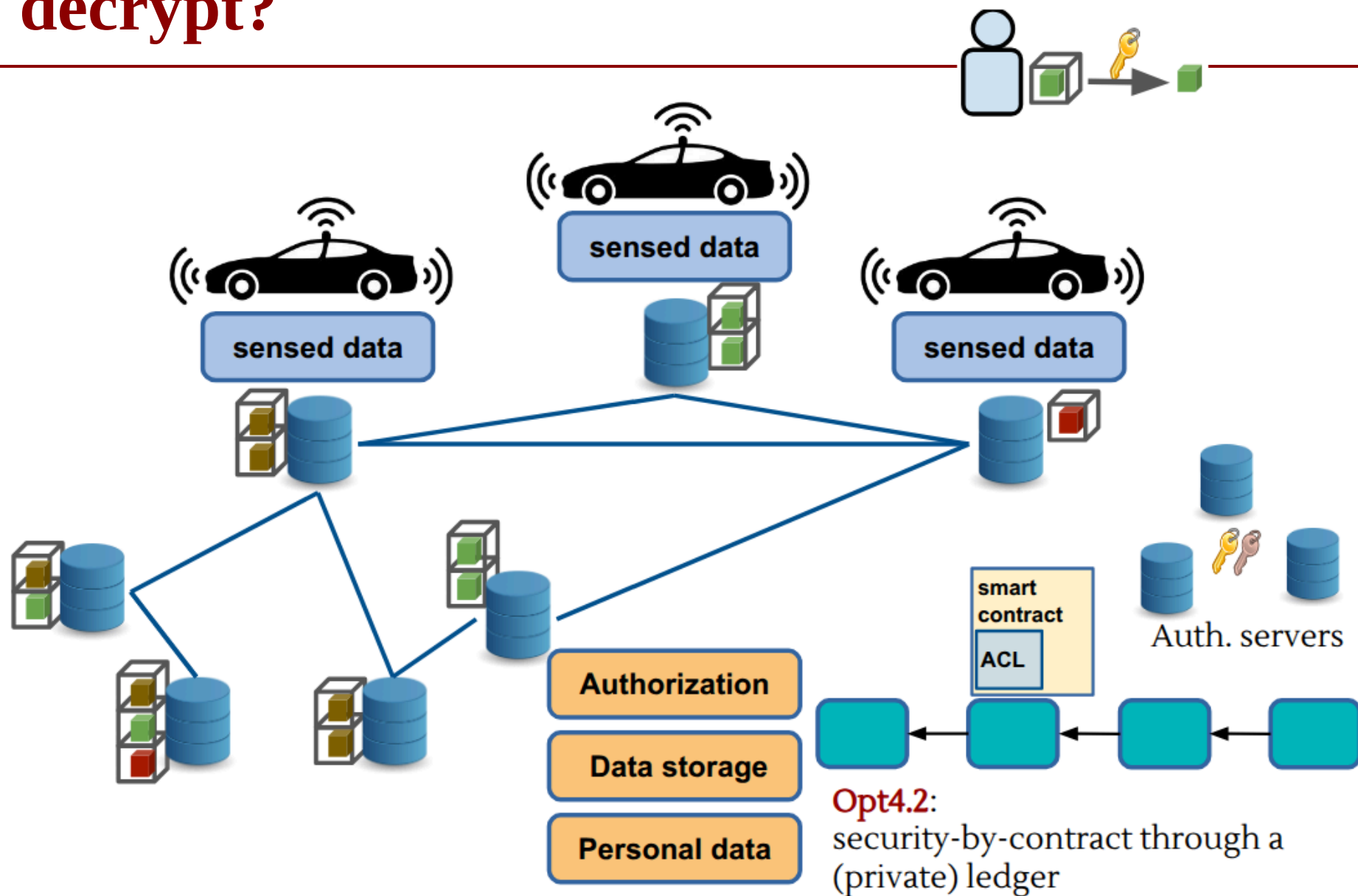
How to decrypt?



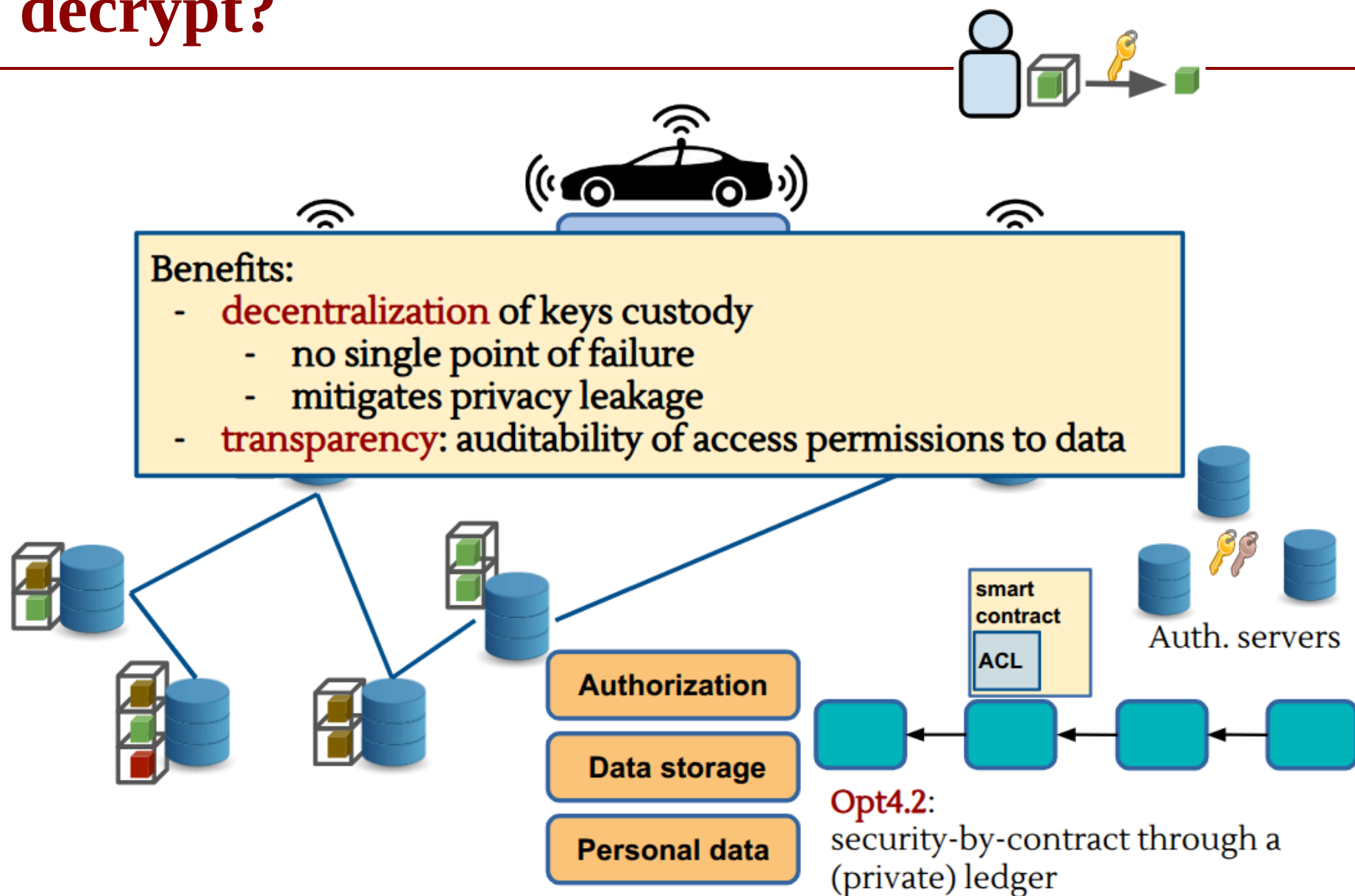
How to decrypt?



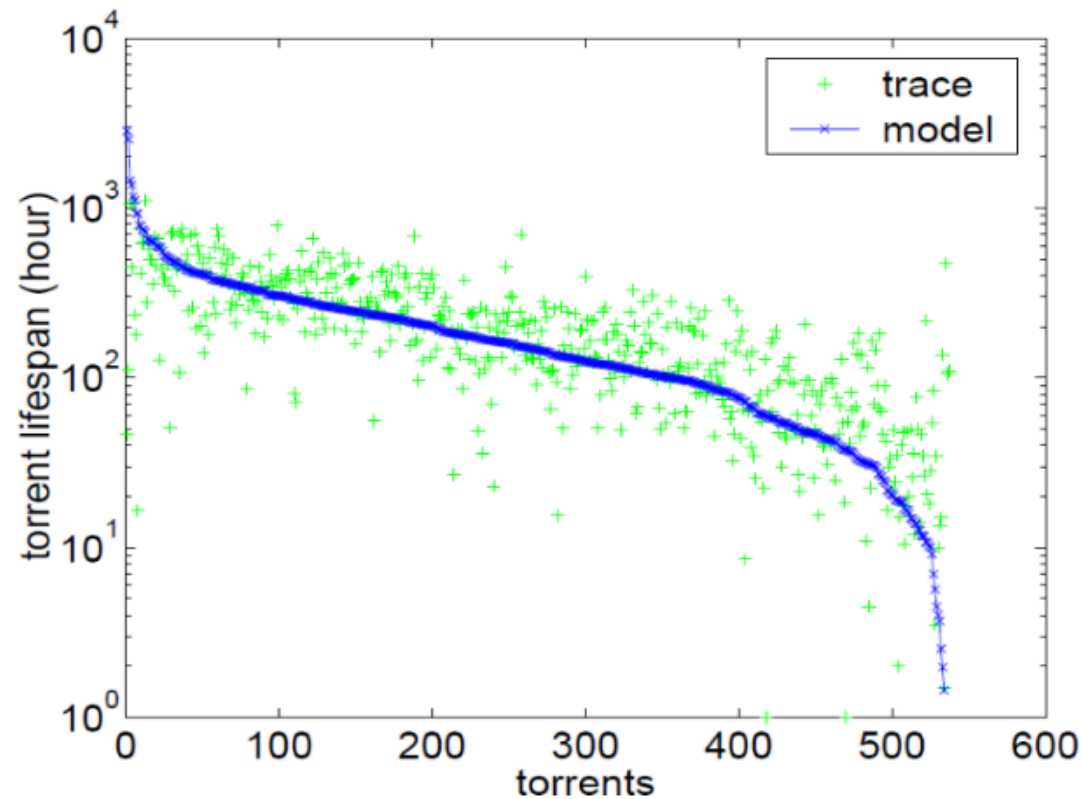
How to decrypt?



How to decrypt?



Data persistence: we'd like to avoid this

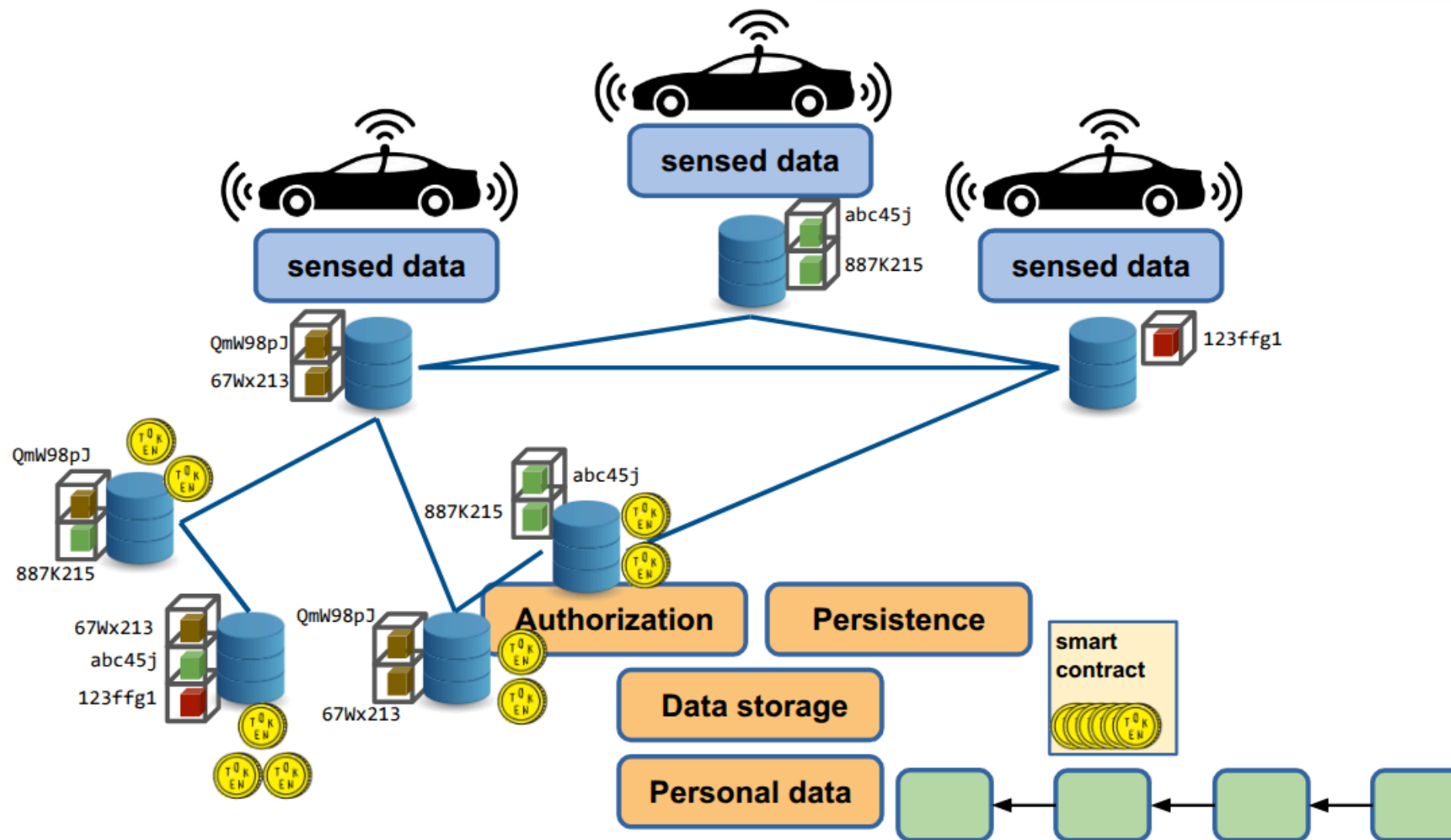


Guo, Lei & Chen, Songqing & Xiao, Zhen & Tan, Enhua & Ding, Xiaoning & Zhang, Xiaodong. (2007). A performance study of BitTorrent-like peer-to-peer systems. Selected Areas in Communications, IEEE Journal on. 25. 155 - 169. 10.1109/JSAC.2007.070116.



Data persistence

Incentives to cooperate:
blockchain based tokens
E.g. Filecoin, Sia, Storj, Swarm (??)



Threat Analysis

Integrity Attacks

- Injection of false mobility data
- Hash substitution attempts
- Replay attacks

Mitigation

- Digital signatures at source
- Hash anchoring on DLT
- Verifiable timestamps



Threat Analysis

Confidentiality Attacks

- Re-identification via mobility traces
- Linkability of pseudonymous identifiers
- Key leakage

Mitigation

- Strong encryption
- Key rotation
- Minimal use of metadata



Threat Analysis

Availability Attacks

- Denial-of-Service against DFS nodes
- Ledger congestion
- Incentive collapse

Mitigation

- Replication policies
- Multi-node anchoring
- Layered fallback mechanisms



Lessons Learned (Crowdsensing)

What really matters

- Data is **distributed**, but risks are **centralized through aggregation**
- **Integrity, privacy, and availability** must be designed together
- Architecture choices directly impact **trust assumptions**

What we need to study

- Secure data sharing
- Distributed systems
- Privacy-preserving techniques (anonymization, aggregation)



Use Case #2: Security Analysis of a Clinical ML System



Scenario

A diagnostic model is trained on data collected from **multiple hospitals**

The model is deployed as a **cloud-orchestrated federated learning system** and accessed through a web API by clinicians

Function: Predict ICU (Intensive Care Unit) readmission risk

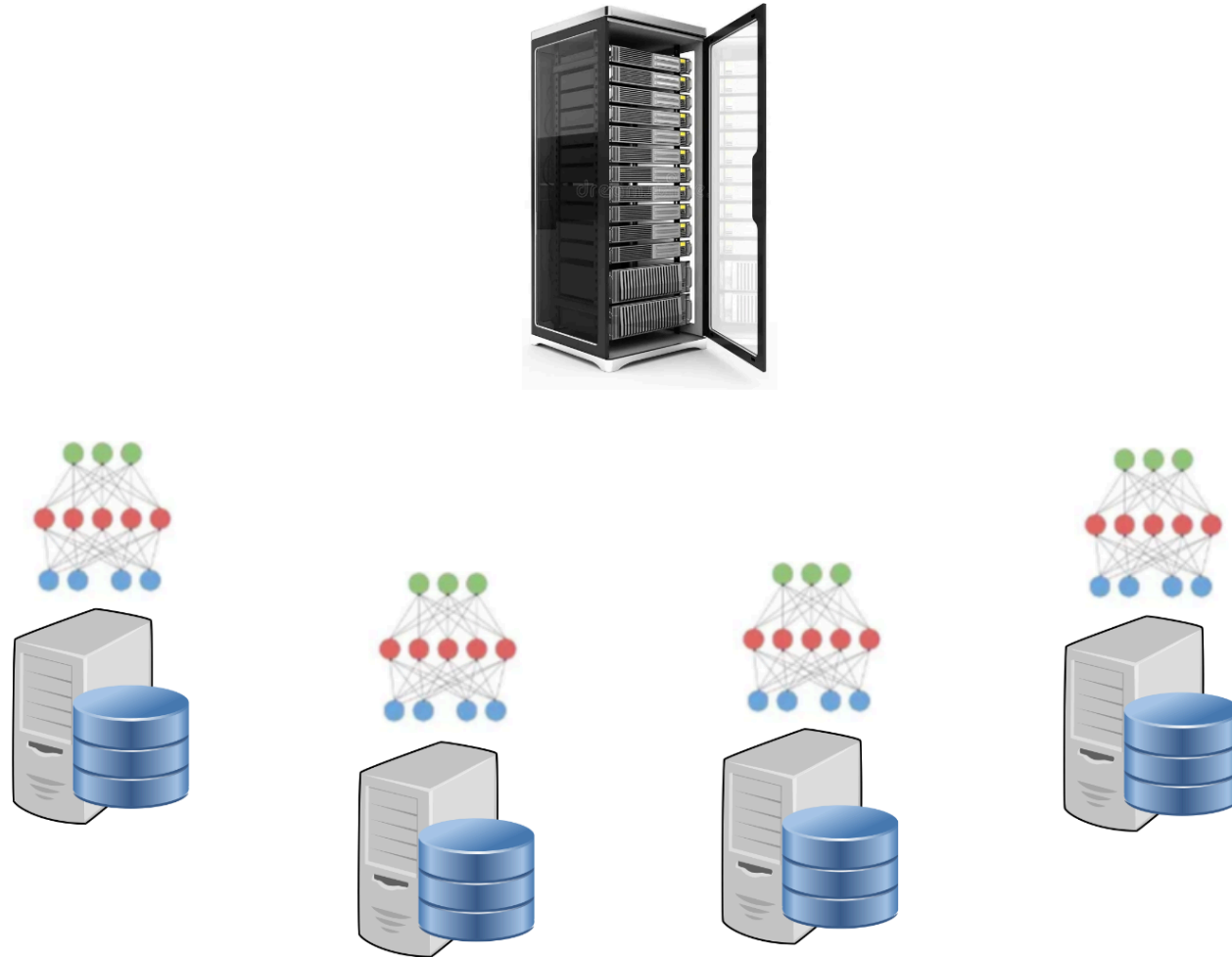
The system must satisfy:

- Regulatory constraints (GDPR compliance)
- Clinical reliability requirements
- Security and resilience against adversarial manipulation



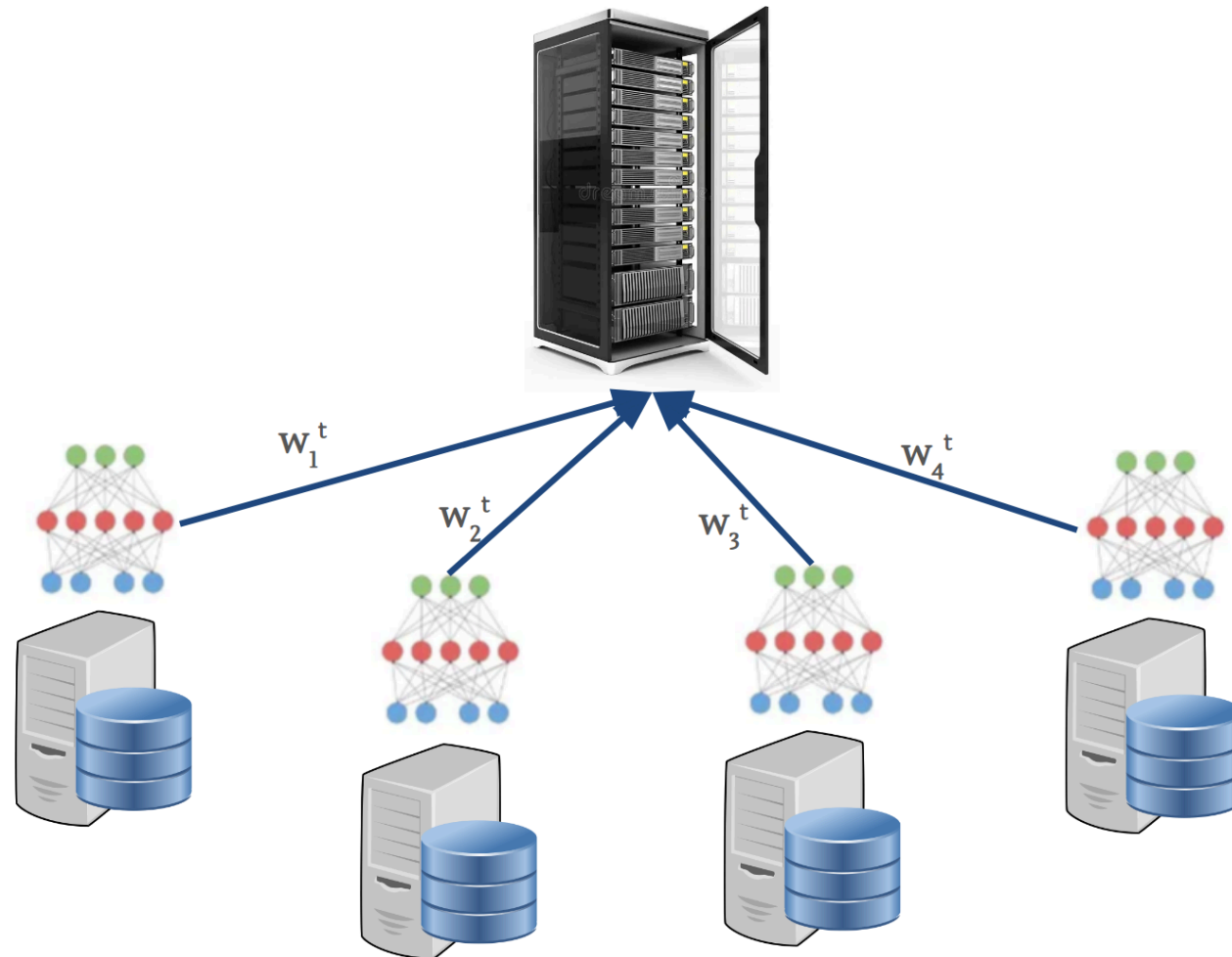
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



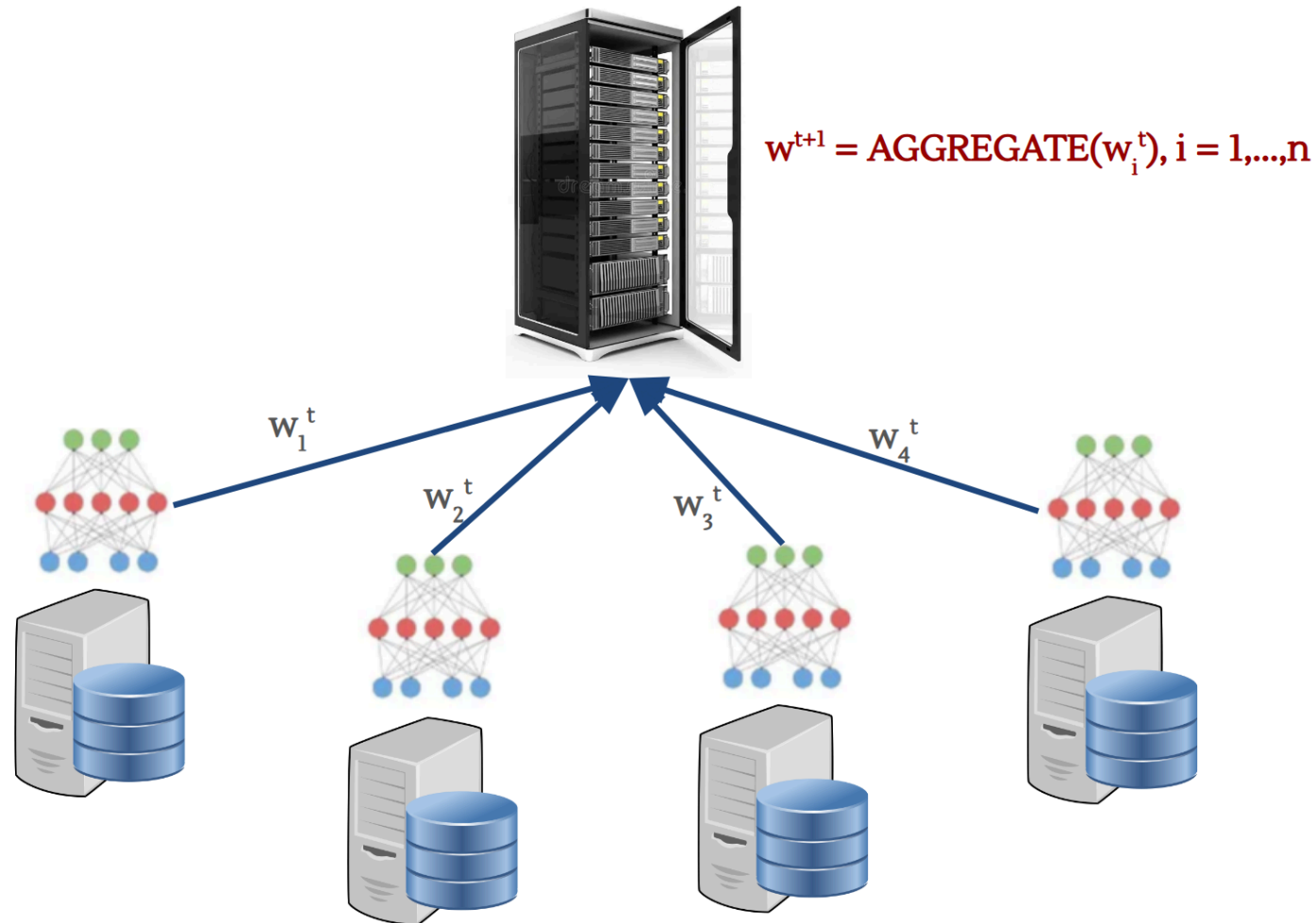
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



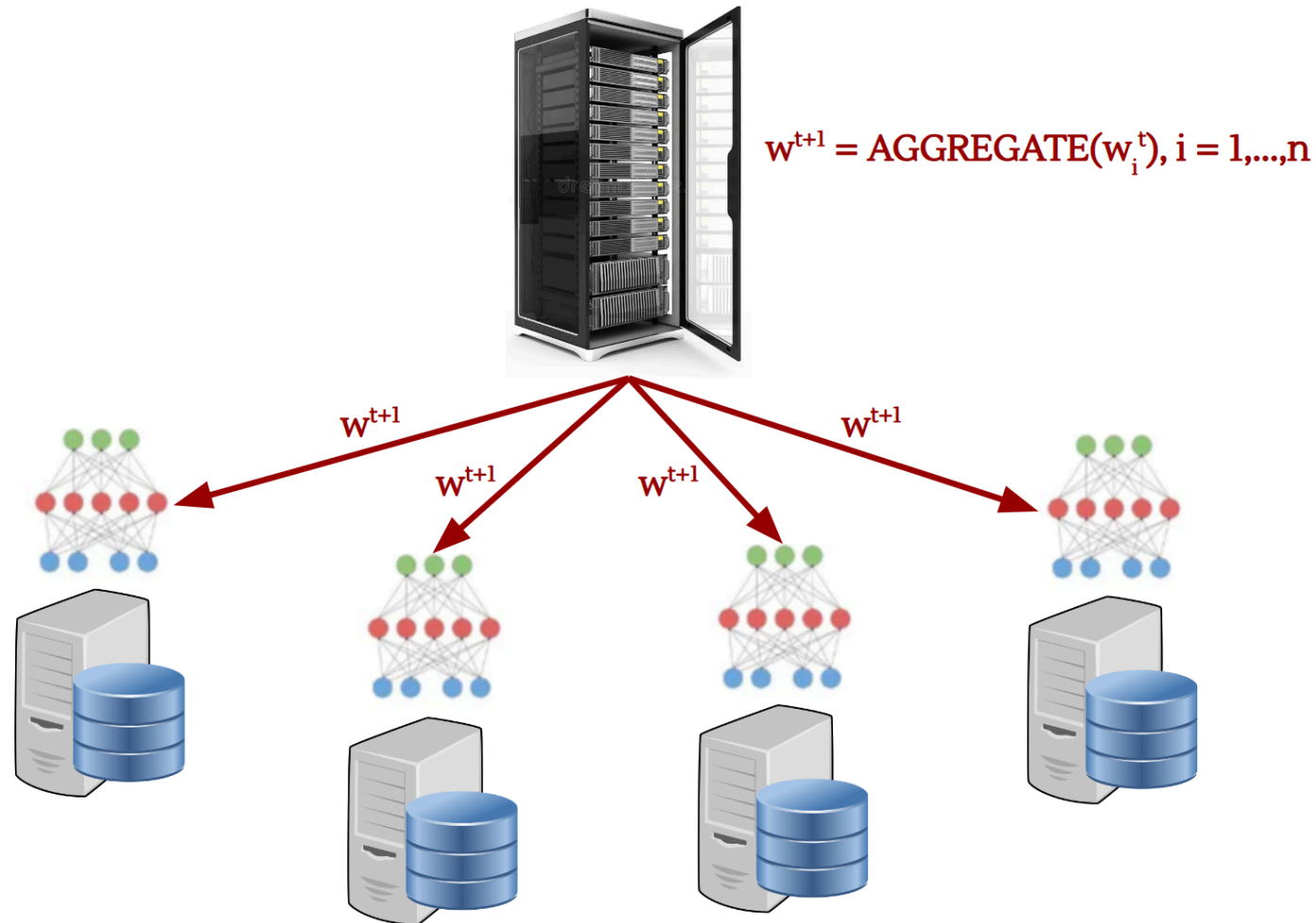
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



System Architecture

Data Characteristics

- Patient records remain local to each hospital
- Model updates (gradients or weights) are transmitted to a central aggregator
- During **training**, labels are produced by external annotation contractors that have access to local data

Deployment

- Cloud-based infrastructure
- API-based access for clinical users



Threat Modeling Framework

We classify threats according to three primary security objectives:

- **Integrity:** correctness of model parameters and outputs
- **Privacy:** protection of patient information
- **Availability:** continuity and reliability of clinical service



Integrity Threat 1: Data Poisoning

Attack Surface

Training labels provided by external contractors



Integrity Threat 1: Data Poisoning

Attack Surface

Training labels provided by external contractors

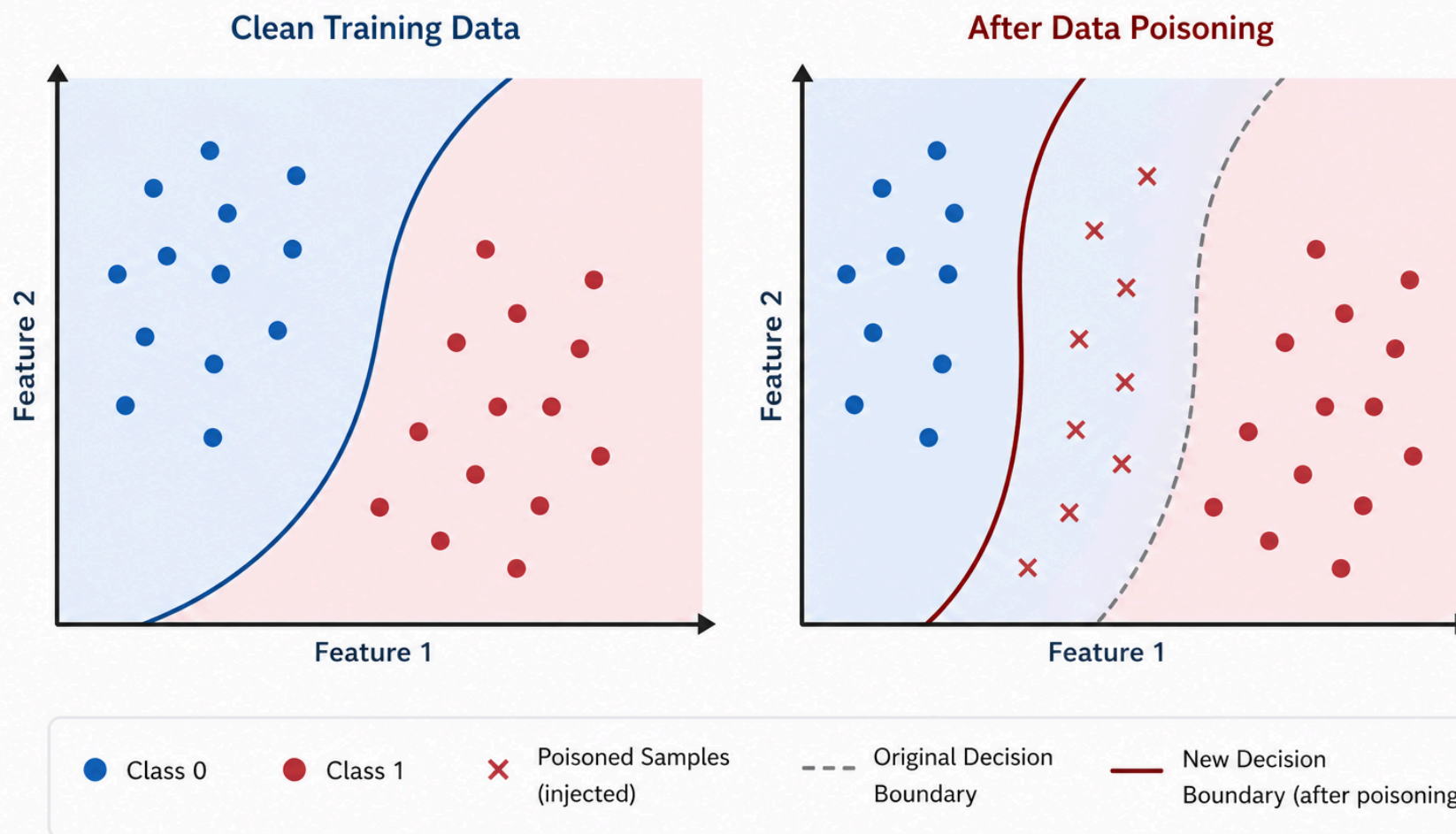
Threat Description

- Incorrect or maliciously **altered labels** degrade model correctness
- Systematic bias may be introduced in specific sub-populations



Integrity Threat 1: Data Poisoning

Effect of Data Poisoning on the Decision Boundary



Integrity Threat 1: Data Poisoning

Measurable Mitigation

Inter-Rater Reliability (IRR) Scoring

- Each record labeled by ≥ 3 annotators
- Discard or re-label samples below a defined agreement threshold

Trade-off

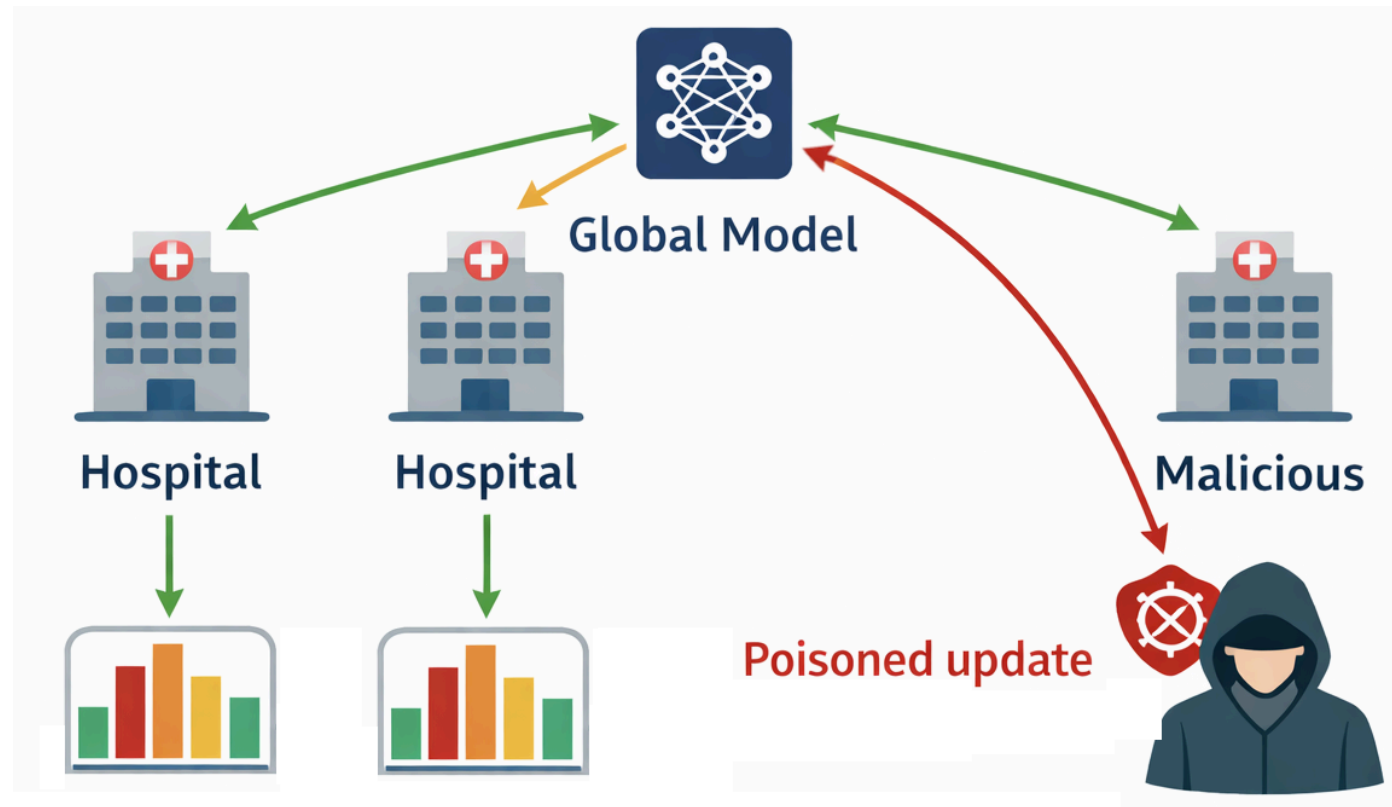
- Increased annotation cost (approximately linear in redundancy factor)
- Increased dataset preparation latency
- Possible reduction in effective dataset size



Integrity Threat 2: Poisoned Federated Updates

Attack Surface

Model updates transmitted by hospitals in the FL protocol



Integrity Threat 2: Poisoned Federated Updates

Attack Surface

Model updates transmitted by hospitals in the FL protocol

Threat Description

A compromised participant sends **adversarial gradients** to bias the global model

Effects may include:

- Targeted backdoor behavior
- Systematic degradation of specific predictions



Integrity Threat 2: Poisoned Federated Updates

Measurable Mitigation

Robust Aggregation Rules

Examples:

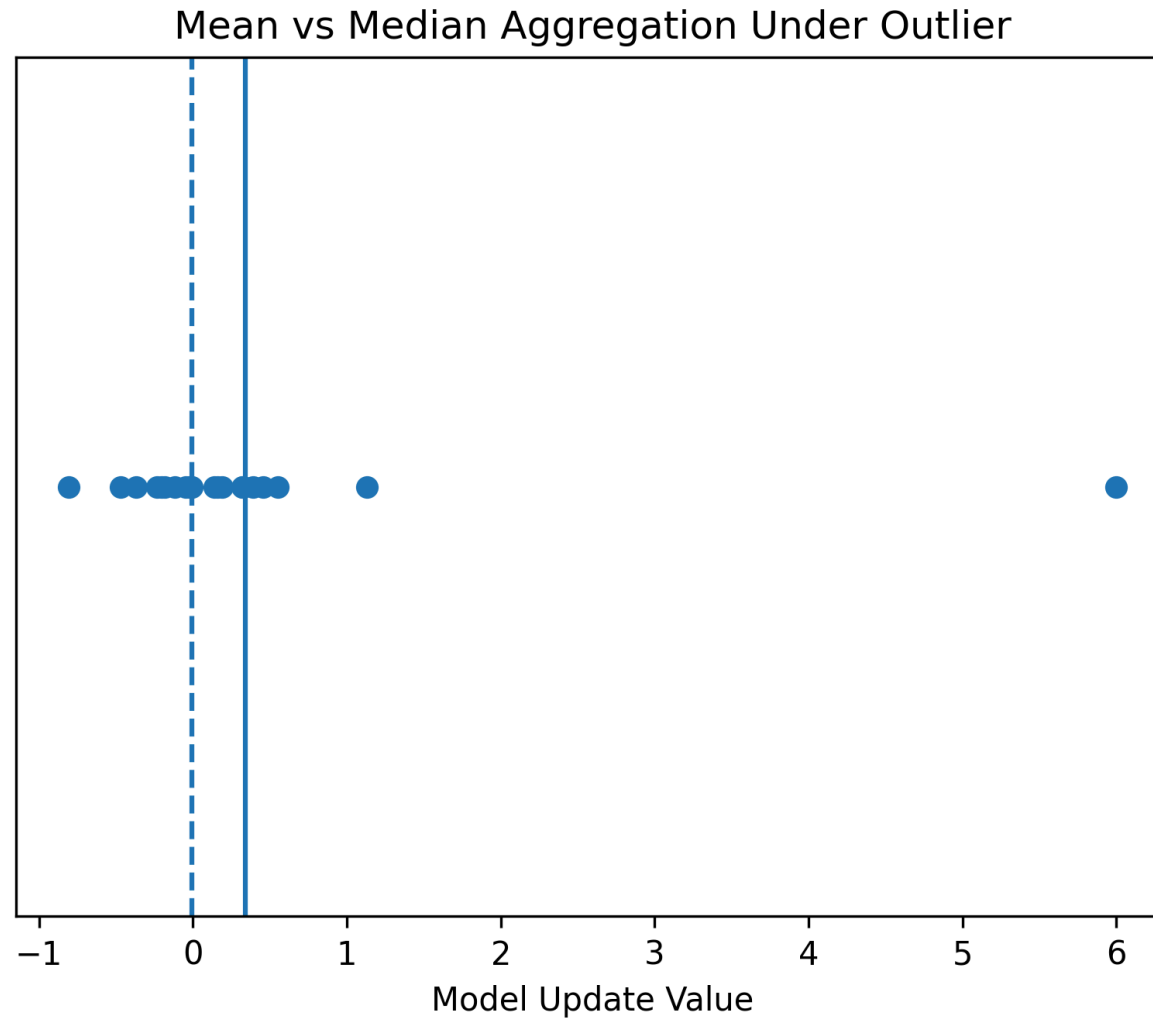
- Krum
- Coordinate-wise median
- Trimmed mean

Metric:

- Bounded influence of outlier updates



Integrity Threat 2: Poisoned Federated Updates



Integrity Threat 2: Poisoned Federated Updates

Trade-off

- Reduced sensitivity to legitimate distributional heterogeneity
- Potential degradation of performance on minority subpopulations
- Increased aggregation complexity



Privacy Threat: Membership Inference

Attack Surface

Public-facing inference API

Threat Description

An adversary queries the model to determine:

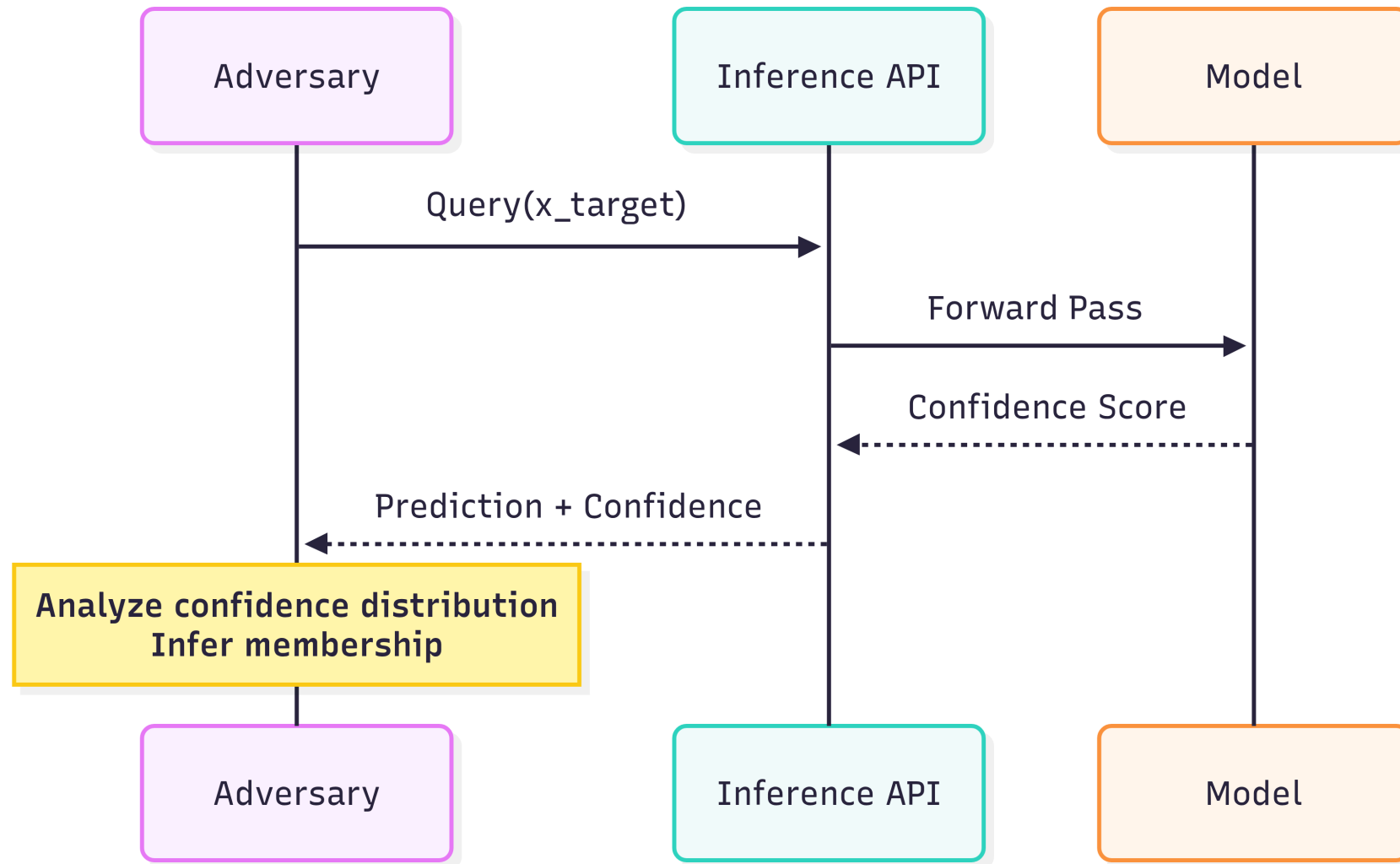
- Whether a specific patient record was used in training
- Whether sensitive attributes can be reconstructed

Relevant attacks:

- Membership inference
- Model inversion



Privacy Threat: Membership Inference



Privacy Mitigation: Differential Privacy

Mechanism

- **Differential privacy (DP)** is a *mathematically rigorous privacy definition* where one dataset vs its neighbor should not significantly change the output distribution
- DP is commonly described via the (ϵ, δ) -condition

Formal Guarantee

For adjacent datasets D, D' differing in one record:

$$P(M(D) \in S) \leq e^\epsilon P(M(D') \in S) + \delta$$



Availability Threat: Adversarial Denial of Service

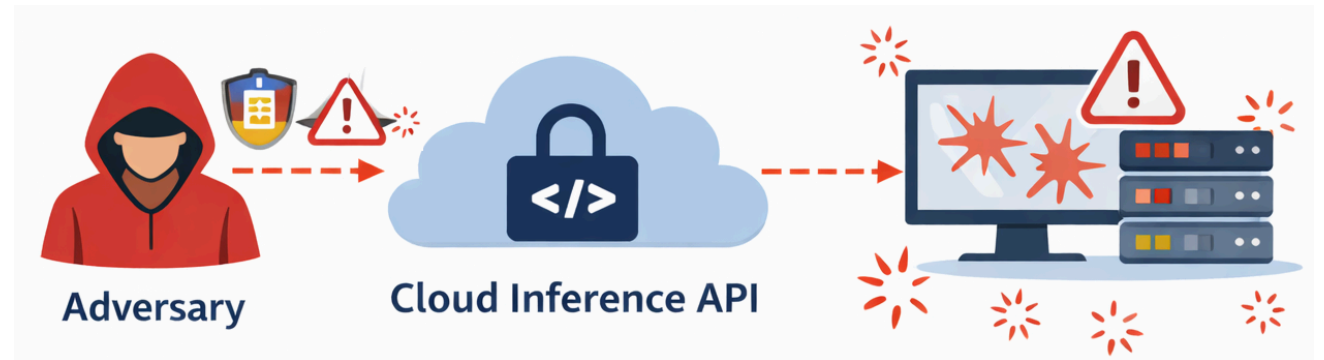
Attack Surface

Cloud inference API

Threat Description

Adversarial inputs crafted to:

- Maximize computational load ("sponge samples")
- Trigger numerical instability
- Cause repeated exceptions



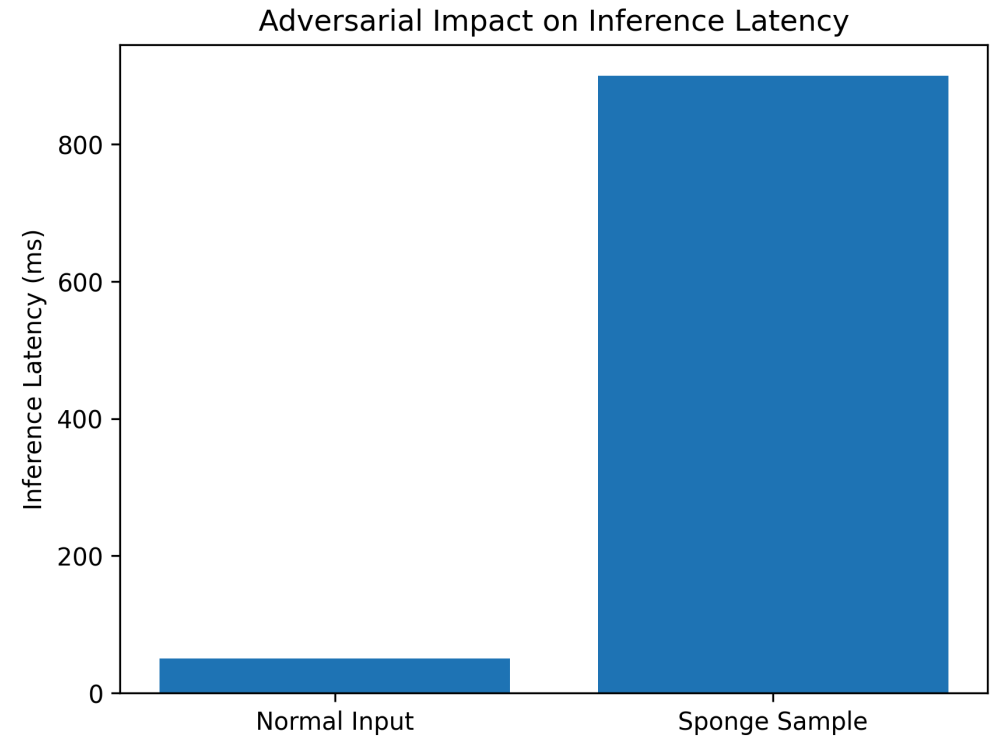
Availability Threat: Adversarial Denial of Service

Effect

- Increased latency
- Degraded clinical workflow
- Potential service outage

Controls

1. API rate limiting per authenticated user
2. Input validation and anomaly detection
3. Resource caps per inference request



Availability Mitigation

Measurable Metrics

- Maximum latency per request
- Rejection rate of anomalous inputs
- Service uptime percentage

Trade-off

- False rejection of legitimate but atypical clinical cases
- Operational friction in emergency contexts



Lessons Learned (Clinical ML)

What really matters

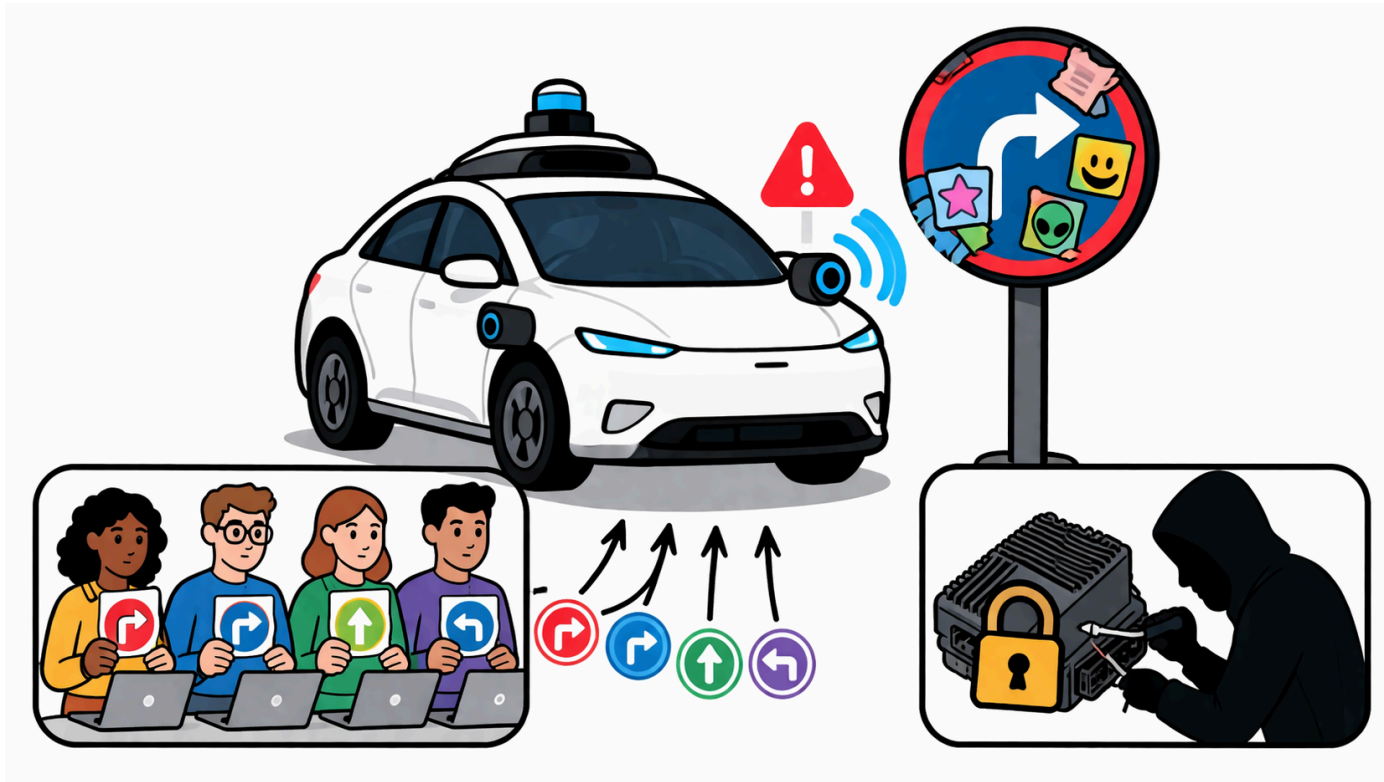
- ML systems introduce **new attack surfaces** (training + inference)
- Security is **data-dependent**, not only system-dependent
- Privacy risks persist even without direct data access

What we need to study

- Trustworthy AI (adversarial ML, poisoning, evasion, inference attacks)
- Differential Privacy
- Federated Learning security
- Evaluation under adversarial settings



Use Case #4: Security Analysis of an Autonomous Perception System



Scenario

A perception model for an **autonomous vehicle** is deployed on an embedded ECU (Electronic Control Unit) with:

- 4 GB RAM
- Limited compute budget
- Infrequent software update capability

The system must remain reliable under:

- Sensor noise and weather variability
- Physical-world adversarial manipulation (e.g., adversarial stickers)
- Supply-chain risks in the annotation pipeline



System Architecture

Function: Traffic sign recognition and scene perception

Deployment Paradigm: Edge inference + selective cloud interaction



System Architecture

Data Pipeline

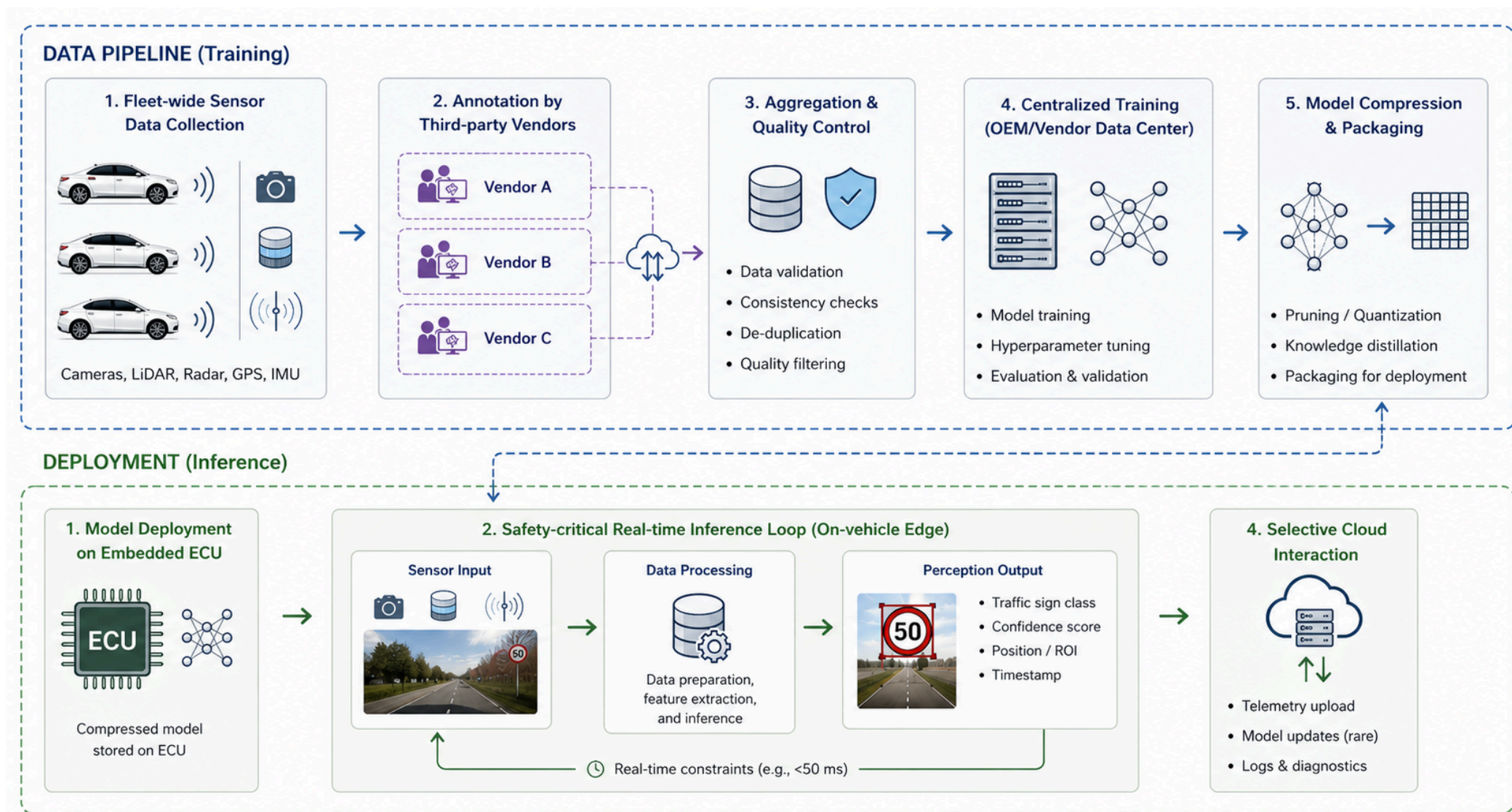
- Fleet-wide sensor data collection
- Annotation performed by three third-party vendors
- Centralized training performed in the vendor data center

Deployment

- Compressed **model deployed** on embedded ECU
- **No** continuous online **retraining**
- Safety-critical real-time inference loop



System Architecture



Threat Modeling Framework

Security objectives:

- **Integrity:** Correctness of model parameters and outputs
- **Availability:** Real-time inference under adversarial conditions
- **Authenticity:** Assurance that deployed model is untampered



Integrity Threat 1

Physical Adversarial Patches

Attack Surface

Road signs modified with localized adversarial stickers



Integrity Threat 1

Threat Model

An adversary applies a perturbation δ such that:

$$\|\delta\|_{\infty} \leq \epsilon$$

but:

$$f(x) \neq f(x + \delta)$$

Note: $\|x\|_{\infty} = \max |x_i|$



Robustness-Capacity Trade-off

Certified robustness typically requires:

- Robustness to $\|\delta\|_{\infty} \leq \epsilon \rightarrow$ deep neural nets
To enforce stability over perturbation regions
- Deep NNs $\rightarrow$ longer training time
Robust training involves additional constraints and computations
- Overfitting avoidance $\rightarrow$ reduced accuracy on clean data
Smoother decision boundaries reduce sensitivity but also precision



Integrity Threat 2

Annotation Supply-Chain Poisoning

Attack Surface

Labels provided by three external vendors

Threat Description

- Systematic label corruption
- Targeted backdoor triggers
- Class-conditional bias injection

Potential consequences

- Misclassification of specific traffic signs
- Hidden backdoor activation



Integrity Threat 2: Mitigation

Controls

1. Redundant Annotation

- Each sample labeled by ≥ 3 annotators

2. Agreement Scoring

- Multi-rater consistency
- Reject samples with consistent labels below threshold

3. Vendor Drift Monitoring

- Per-vendor confusion matrix tracking
- Statistical deviation detection



Integrity Threat 2

Trade-off

- Increased annotation cost
- Dataset size reduction
- Increased preparation latency



Availability Threat

Adversarial Sensor Inputs

Attack Surface

Camera and LiDAR streams

Threat Description

Inputs crafted to:

- Trigger worst-case compute paths
- Induce latency spikes
- Cause frame drops beyond control deadlines

Consequence:

- Missed real-time constraints
- Unsafe control transitions



Availability: Mitigation

Controls

1. Input validation and preprocessing constraints
2. Worst-case latency profiling
3. Resource isolation at process level

Metrics

- Maximum inference latency (ms)
- 99.9th percentile latency under adversarial load
- Frame drop rate

Trade-off:

- False rejection of legitimate rare scenarios



Authenticity Threat

Model Tampering on ECU

Attack Surface

Model binary stored in on-device flash memory

Threat Description

- Unauthorized firmware modification
- Parameter replacement
- Backdoored model injection



Authenticity Mitigation: Trusted Execution

Controls

1. Secure Boot

- SHA-256 integrity check of firmware at load time

2. Trusted Execution Environment (TEE)

- Isolated inference runtime
- Remote attestation

Metrics

- Integrity verification

Trade-off

- Increased startup latency
- Hardware cost overhead



Lessons Learned (Autonomous Systems)

What really matters

- The physical world becomes an **attack surface**
- Constraints (latency, memory) limit defenses
- Supply chain is part of the threat model

What we need to study

- Robustness of ML models
- Secure deployment (TEE, secure boot)
- Real-time constraints and worst-case analysis
- Data pipeline security



Use Case #5: Overlay Networks of Interacting AI Agents



Scenario

We consider a system composed of:

- N AI agents
- Each agent runs a Small Language Model (SLM)
- Agents interact over an overlay graph $G = (V, E)$
 - Nodes V : AI agents
 - Edges E : communication channels

The system is **fully decentralized**

- System behavior is no longer local, but **emergent and topology-dependent**



Why this is relevant

Future AI infrastructures will be:

- Distributed
- Autonomous
- Cooperative
- Heterogeneous

Examples:

- Personal agents interaction
- Distributed fraud detection
- Autonomous vehicle swarms
- Cyber threat intelligence sharing
- Edge IoT infrastructures



Why this is relevant



Agent Interaction Model

Each agent maintains a state s_i^t

Update rule:

$$s_i^{t+1} = F_i (s_i^t, \{s_j^t : j \in \mathcal{N}(i)\})$$

where:

- $\mathcal{N}(i)$ = neighbors of agent i
- F_i implemented via SLM reasoning

Security implication

Local compromise → Global instability



Threat 1: Compromised High-Centrality Agent

Assume node k has:

- High degree centrality
- High betweenness centrality
- High eigenvector centrality

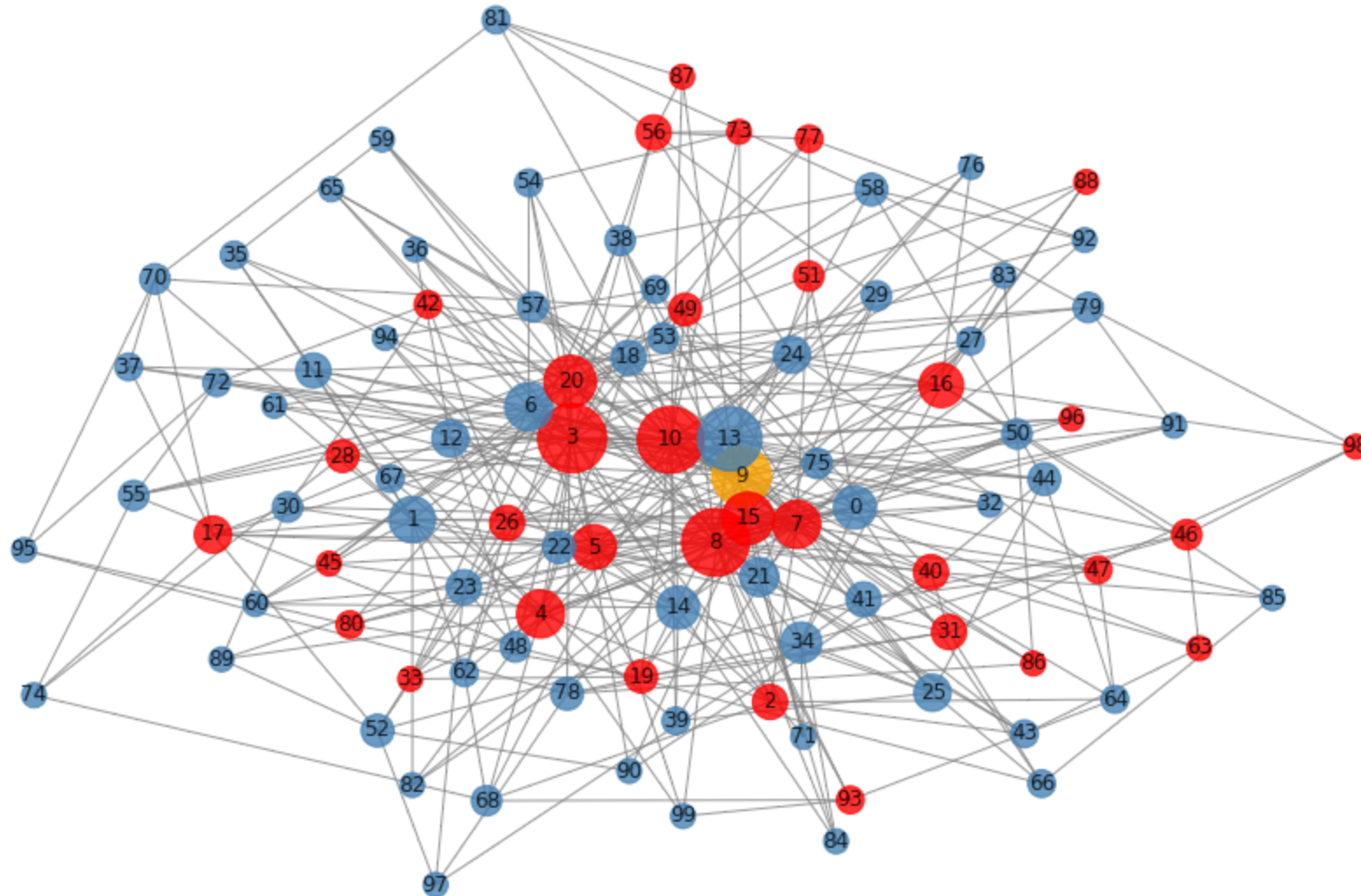
If compromised: $s_k^t \rightarrow s_k^t + \delta$

Impact

- Rapid misinformation propagation
- Cascading behavioral drift
- System-wide instability



Threat 1: Compromised High-Centrality Agent



Threat 2: Coordinated Minority Attack

Subset $S \subset V$ behaves adversarially

Effects:

- Biased consensus
- Polarization
- Decision boundary shift

Critical question:

Is there a **phase transition threshold** for successful attack?



Threat 3: Topology-Driven Amplification

Certain graph structures amplify perturbations:

- Scale-free networks
- Highly clustered communities

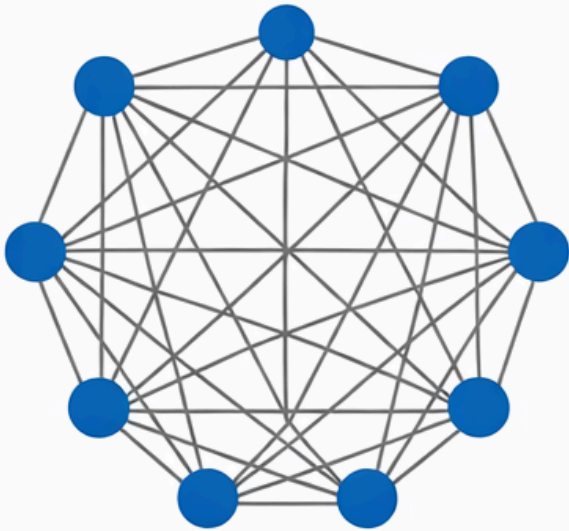
Small perturbations may propagate exponentially

Security is topology-sensitive

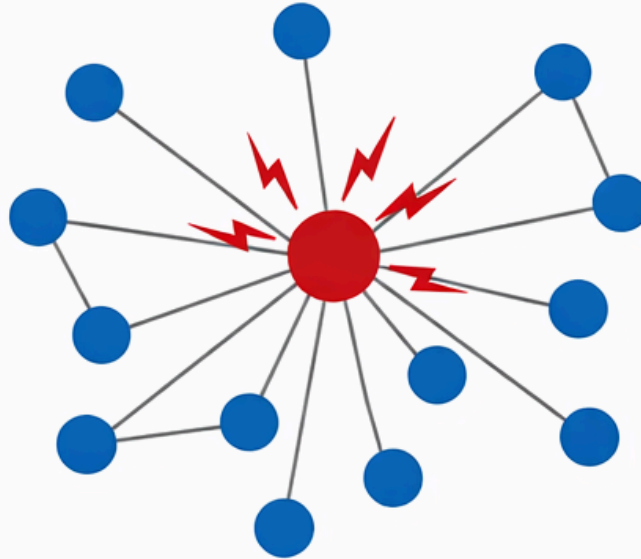


Threat 3: Topology-Driven Amplification

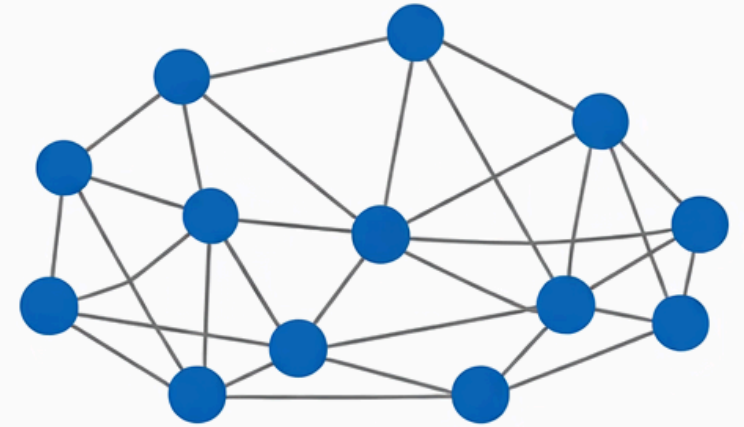
Fully Connected



Scale-Free: Hub Compromised



Small-World



Why Network Theory is Necessary

This is not a classical ML security problem, we are dealing with a **dynamical system over a graph**

We must analyze:

- Degree distribution
- Network connectivity
- Clustering coefficient
- Percolation thresholds



Mitigation 1: Trust-Weighted Aggregation

Modify update rule:

$$s_i^{t+1} = F_i \left(s_i^t, \sum_{j \in \mathcal{N}(i)} w_{ij} s_j^t \right)$$

Where:

- w_{ij} = trust weights
- dynamically updated

Goal: reduce adversarial influence



Mitigation 2: Topology Engineering

Design graph to improve robustness:

- Increase net connectivity
- Limit hub dominance
- Avoid extreme scale-free structures
- Introduce redundancy

In this scenario, security is partially a graph design problem



Mitigation 3: Byzantine-Resilient Consensus

Use robust aggregation:

- Median-based update
- Trimmed mean
- Outlier rejection

Goal: Bound adversarial impact even if fraction f of nodes is malicious



Mitigation 4: Influence Monitoring

Compute periodically:

- Node centrality metrics
- Influence functions
- Local divergence indicators

Early detection of:

- Behavioral drift
- Coordinated anomalies



Lessons Learned (AI Agent Networks)

What really matters

- Security is **emergent and topology-dependent**
- Local compromises can cause **global failures**
- Trust must be **distributed and dynamic**

What we need to study

- Network theory and graph robustness
- Distributed consensus under adversaries
- Trust and reputation systems
- Dynamical systems and stability analysis



Part III: Some problems and techniques we have identified so far



Foundational Terms - CIA



Foundational Terms - CIA + ML Extension

The classic **CIA triad** remains the backbone:

Property	Classic Definition	ML Extension
Confidentiality	Data visible only to authorized parties	Training data, model weights, inference outputs
Integrity	Data/system not modified without authorization	Model predictions not manipulated by adversary
Availability	System accessible when needed	Model not degraded by denial-of-service or poisoning



Threat Actors & Their Capabilities

We classify adversaries along two axes:

Knowledge:

- **White-box:** full access to model architecture, weights, training data
- **Gray-box:** partial knowledge (architecture known, weights unknown)
- **Black-box:** query-only access (most realistic in production)

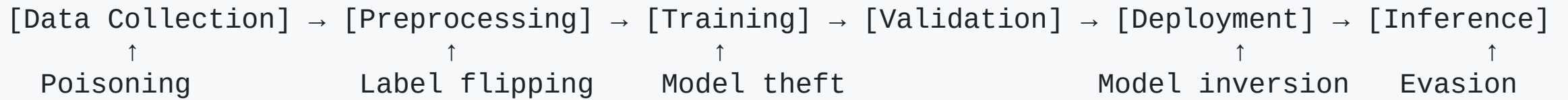
Goals:

- **Integrity attacks:** cause misclassification
- **Privacy attacks:** extract sensitive information from model
- **Availability attacks:** degrade model performance
- **Backdoor attacks:** embed hidden behavior triggered by a specific input pattern



Threat Modeling for ML Systems

The ML pipeline as an attack surface:



AI for Cybersecurity

to conclude ... since we gave it for granted



AI as a Tool for Defense Cybersecurity

Application	ML Approach	Key Challenge
Intrusion Detection	Anomaly detection, sequence models (e.g., LSTM)	Maintaining low false positives under high-volume traffic
Malware Classification	Static/dynamic analysis, GNNs	Robustness to adversarial evasion techniques
Fraud Detection	Supervised learning (e.g., gradient boosting), online learning	Drift due to evolving attacker adaptation
Behavioral Analytics	Sequence modeling, user embeddings	Preserving user privacy while modeling behavior
Threat Intelligence	NLP on CTI reports, knowledge graphs	Data quality and noise in OSINT



Some Recommended Resources to Start

Primary references:

- Goodfellow et al. — *Deep Learning* (MIT Press) — Ch. on Robustness
- Dwork & Roth — *The Algorithmic Foundations of Differential Privacy* (2014)
- Papernot et al. — *SoK: Security and Privacy in ML* (IEEE EuroS&P 2018)

Tools & frameworks:

- `CleverHans` , `ART` (IBM) — adversarial attack/defense libraries
- `Opacus` (PyTorch), `TF Privacy` — differential privacy
- `PySyft` — federated learning & SMPC
- `FATE` — industrial federated learning framework





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Prof. Stefano Ferretti

**Department of Computer Science and
Engineering**

University of Bologna

s.ferretti@unibo.it